

C语言与程序设计

The C Programming Language

第8章 指针

王多强

1097412466

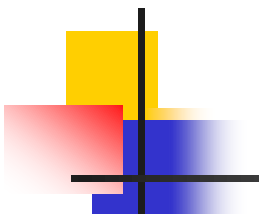
dqwang@hust.edu.cn

华中科技大学计算机学院



本章主要内容

- 8.1 指针的概念与指针的使用
- 8.2 指针运算
- 8.3 指针作为函数的参数
- 8.4 数组的指针表示
- 8.5 指针数组
- 8.6 带参数的main函数*
- 8.7 指针函数
- 8.8 函数的指针*
- 8.9 restrict和_Atomic类型修饰符 **



指针是C语言最具特色的内容。能否正确理解和使用指针是是否熟练掌握C语言的一个标志。同时，指针也是C语言中最为困难的一部分。

要想理解指针，先记住一句话：

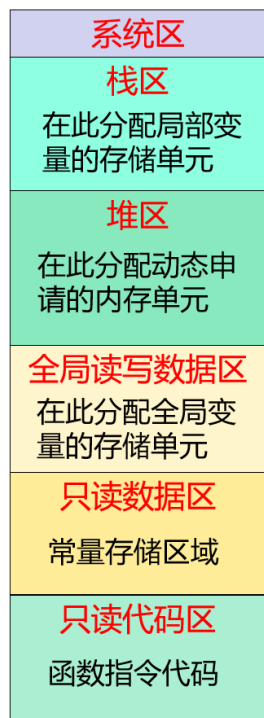
指针就是内存单元的地址

8.1 指针的概念

8.1.1 地址和指针

1、指针即地址

程序运行时，程序的**变量**、**常量**、**指令代码**等都要调入内存，系统为每个对象分配若干字节的存储空间存储其值（或内容）。



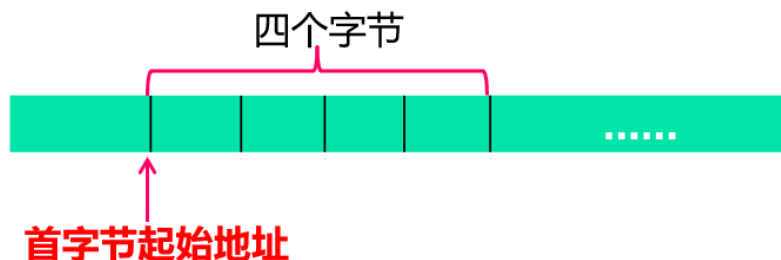
系统内存区域示意图

- ◆ 变量在**可读写数据区**（全局区、栈区、堆区）分配存储单元，内容可被修改；
- ◆ 常量在**只读数据区**分配存储单元，只能读，不能被修改；
- ◆ 函数在**代码区**分配存储空间存储其指令代码，只能读，不能被修改；

操作系统对内存中的每个字节都编制了一个唯一且不变的位置编号，称为**字节的地址**。**字节地址由系统统一管理，与系统中运行的程序无关。**

一个程序运行时，系统会为程序中的**变量、常量、函数等对象**分配存储单元。**将各类对象所占字节空间的第一个字节的地址称为该对象在内存中的地址**，简称为**对象的地址**，亦称为**对象的指针**——亦即：**指针即地址、地址即指针**。

如：单个变量 `int x`；内存中占4个字节



2、取地址运算符：&

可以用**取地址运算 &** 获取变量的地址，如：`int x; &x`

- ◆ **& 只能作用在变量上**，不能对表达式或常量取地址。

如：`int x; &x` ✓ `&(x+y)` ✗ `&"Wuhan"` ✗

注：字符串常量在只读数据区有自己的存储单元，所以有自己的地址，可以在程序里使用（见字符指针的相关内容）。但**数值型常量和字符型常量**一般仅存在于机器指令中，没有独立存储单元，不能对其取地址。

`&5` ✗ `&1.0` ✗ `&'a'` ✗

- ◆ 数组名本身代表数组的地址，**不能用&取数组名的地址，或试图改变数组名的值。** `int a[20];` `&a` ✗ `a=&x` ✗

➤ 但**数组元素是“变量”**，可以取其地址。 `&name[0]` ✓

◆ 地址是一种什么类型的数据？

字节地址是一个 $0 \sim 2^n - 1$ 范围内的整数，且

(1) 均为非负数：没有负数地址；

(2) **所有字节地址的编码长度一样**：都等于机器字长。如在32位计算机系统中，地址编码就是**32bit**的整数。

所以，**地址可被视为一种无符号整数 (unsigned int) 。**

但同时，**地址又是一种特殊的数据**，不能简单作为普通的无符号整数使用，而是要遵循**地址型数据特有的一些运算规则**，因此地址数据可以看作是一种**运算受限的无符号整数**。

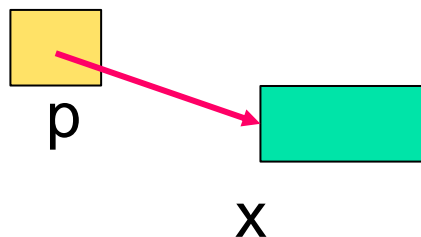
3、指针变量

地址作为一种数据，也可以**定义变量来保存它**。用来存放地址数据的变量称为**指针变量**。

4、指针起到指示器的作用

地址或指针的含义就是**指示器**，指明了一个对象在内存中的位置。

- ◆ 故一般称一个**对象的指针（地址）指向了该对象**；
- ◆ 如果一个指针变量 **p** 中存放了另一个对象 **x** 的地址，一般也称 **p 指向了 x**。示意图可表示为：



8.1.2 指针变量的声明、存储、赋值和间访

1、指针变量的声明

声明指针变量的一般形式：

数据类型名 * **标识符1**, * **标识符2**, ..., * **标识符n**;

如：**int** * **p**; 或 **char** ***pc**, ***ps**; 或 **float** ***pf**;

这里，

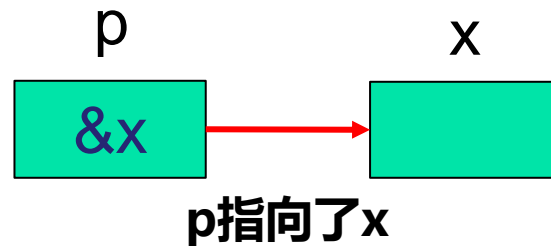
- *：是**指针说明符**，用于将后面的标识符说明为**指针类型变量**。
- **数据类型名**：称为指针的**基类型**，指**指针所指对象**的数据类型。
- **标识符1,..., 标识符n**：指针变量的名字，**合法的标识符**。

2、令指针变量指向一个对象

最直接的方式：**取被指对象（变量）的地址为指针变量赋值：**

如： `int x, *p;`

`p = &x;`



- ◆ 只能用**基类型**相同的地址值为指针变量赋值。

如： `char *pc;` `pc = &x;` ❌ 编译器报错

- ◆ **指针变量初始化**：在定义指针变量的同时为指针变量赋初值称为指针变量的初始化。

如： `int x, *p = &x;`

3、指针类型

对一种数据类型 **T**，可以把 **T *** 看作是 **T** 的一种扩展类型，称为 **T 的指针类型**，简称为 **T * 类型**。

- ◆ 若有 **T *p**，则可将指针变量 **p** 视为是 **T * 类型** 的变量。

同时可引申出：

- ◆ 对 **T 类型** 的变量 **x**，**x 的地址** 的类型是 **T *** 的。

如：**int x**，则 **x的地址** 就是 **int *** 类型的

- ◆ 取变量地址运算 **&x** 的运算结果是 **x 的地址**，所以 **表达式 &x 的类型是 T * 的**。

4、指针变量的存储

指针变量是保存地址型数据的变量，因此**指针变量也有自己的内存单元**。

由于**地址型数据**是一种等于系统机器字长的“**无符号整数**”，所以：设系统的机器字长是N字节（如32位系统，机器字长是**4字节**）。则

- ◆ 所有（字节）地址都是**4字节长度**的“**无符号整数**”。
- ◆ 所有**指针变量的数据长度也都相同**，等于4字节。

—— 也就是，**定义一个指针变量，系统就为其分配4字节的存储空间**，与其基类型无关。

如: `int x=0x1234;`

`int *pi = &x;`

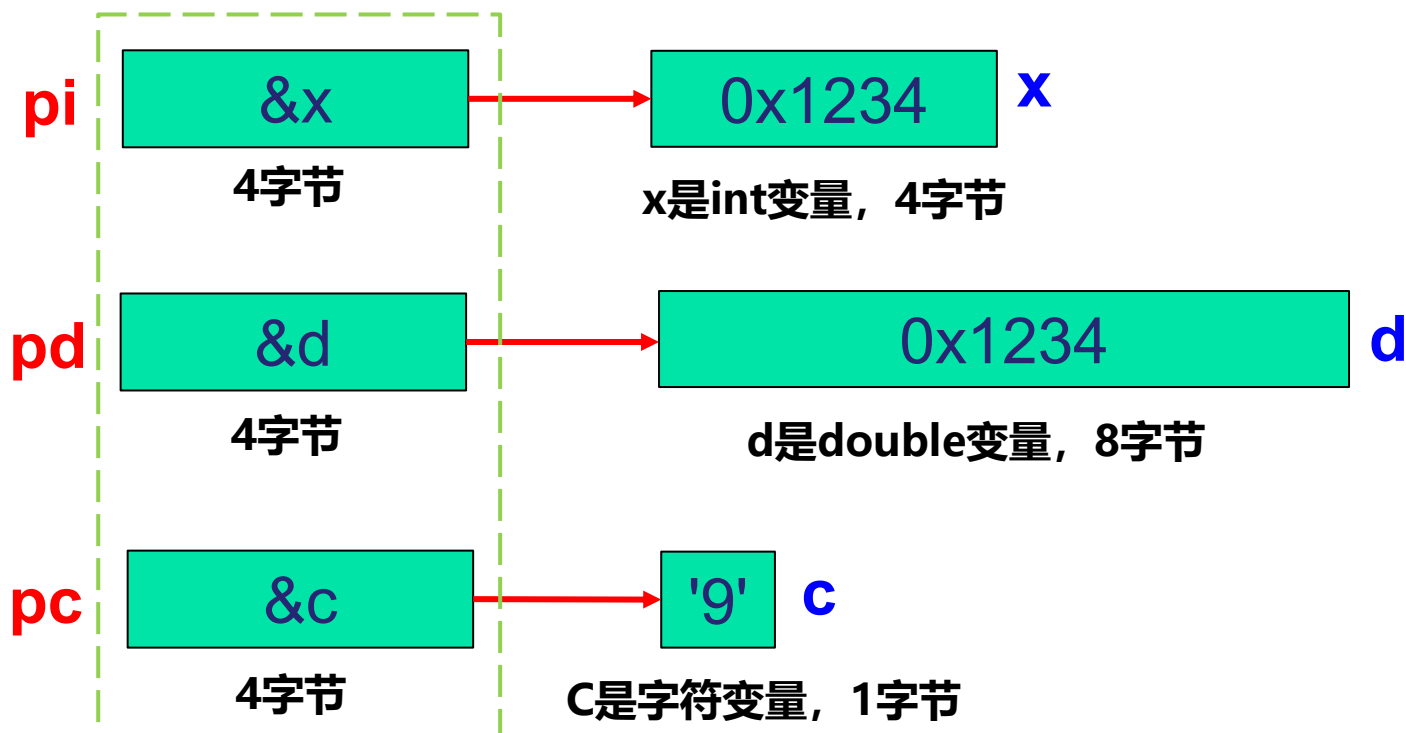
`/*pi是整型指针变量*/`

`double d=0.1, *pd=&d;`

`/*pd是双精度浮点型指针变量*/`

`char c='9', *pc=&c;`

`/*pc是字符指针变量*/`



指针变量都是4字节的

5、指针变量的赋值运算

- 只能用 **(基) 类型相同** 的**地址值** 为指针变量赋值。

如: `int x, *p=&x;` ✓

注: `int *p=&x, x=10;` ✗

`char *pc=&x;` ✗

任何符号都要先声明后使用

- 两个**同类型**的指针变量之间可以赋值;

但不同类型的指针变量之间不能相互赋值。

如: `int *p2;`

`p2=p;` ✓

`pc=p;` ✗

编译器报类型不匹配的错误

- ◆ **类型不一致时**：可以将一种类型的地址通过**类型强制**，转化成另一种类型的地址赋给相应的指针变量 —— 此时，**地址值不变，只是解释指针所指对象时，针对的对象类型不同**。如：

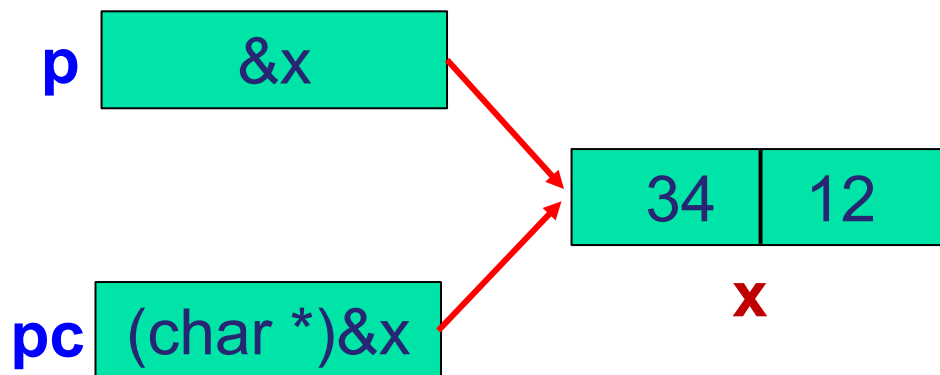
```
short int x=0x1234, *p=&x;
```

```
char *pc;
```

```
pc=&x; ❌
```

```
pc=(char *)&x; ✅
```

```
pc=(char *) p; ✅
```



注：此时，`p` 和 `pc` 的**值一样**，都是 `x` 第一个字节的地址，但**二者**
的类型不同，一个是 `short int*`，另一个是 `char *`，由此会
引起应用上的差异（见后面的内容）。

注：一方面，任何变量一经创建，其位置是不变的，其地址也就不会变动。另一方面，**指针变量保存其它变量的地址，它们的值是可以改变的：**

将指针变量的值从一个对象的地址改为另一个对象的地址时，会使其指向不同的对象。但这不代表移动了哪个对象的位置，**只是改变了指针的指向，而不会改变之前和现在所指的对象本身。**

如：short int x=0x1234, y=0x5678;

short * **p**;

时刻1： **p=&x;** /*p指向x*/ **p**

&x	→	34	12
----	---	----	----

x

时刻2： **p=&y;** /*p指向y*/ **p**

&y	→	78	56
----	---	----	----

y

6、对于一维数组

如： **T x[5]** （T是数组的类型，具体如 **int x[5];**）

(1) **数组中的每个元素都是T类型的变量，可以取其地址，**

如： **&x[0]** **&x[3]** **&x[i]**

而且**每个元素的地址都是 T * 类型的。**

(2) **一维数组的数组名是指向数组第一个元素的指针，其值等于其第一个元素的地址值，类型是 T * 类型。**

(3) **一维数组的数组名也代表整个数组的起始地址，故一维数组起始地址的类型也是 T * 类型的，与元素的地址类型一致。**

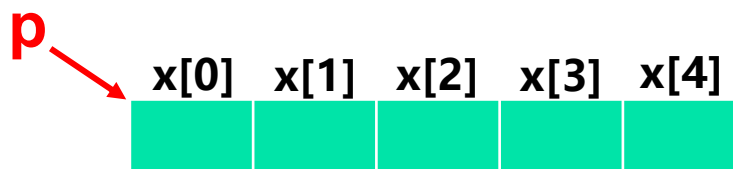
(注：对多维数组而言，数组起始地址的类型与其元素的地址类型不同)

(4) 可用一维数组的数组名为同类型的指针变量赋值。

- ◆ 赋值后，指针变量指向一维数组的第一个元素。

如： `int x[5];`

`int *p = x;`



(4) 也可以用一维数组任何元素的地址为指针变量赋值。

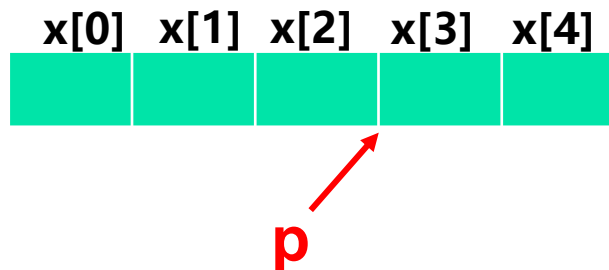
- ◆ 从而使指针变量指向的这个元素：

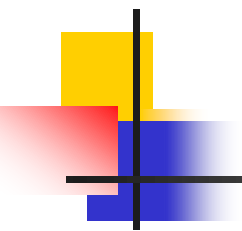
如： `p = &x[3];`

使 `p` 指向数组 `x` 的第四个元素 `x[3]`。

`int x[5];`

`int *p;`





注：**数组名是地址常量**，仅代表数组的起始地址，它指向数组的第一个元素。但**不能为数组名赋值“而使其指向其它元素”**。

如：**`x=&x[3];`** **×**

8.1.3 指针变量的间访运算

- ◆ 把利用**变量名**访问变量的方式称为对变量的**直接访问**；
- ◆ 通过**变量的地址**访问变量称为对变量的**间接访问**，简称**间访**。

1、间访运算的一般形式是：

*** 指针变量**

称为**间访表达式**

如： ***px**

```
int x, *px = &x;
```

*****： **间访运算符**，单目运算符，第2优先级，右结合。

含义： **引用指针所指的对象**。

语法层面， **间访表达式等价于指针所指对象实体**，可以基于指针间访表达式读取对象的值或为其赋值。

如: `int x=0x1234, b;`

`int *p1=&x, *p2=p1;` /*通过赋初值, p1、p2都指向x*/

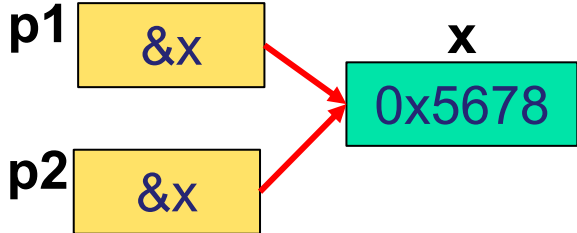
`char c='9', *pc=&c;`

◆ `b=*p1;` 访问p1, 引用p1所指的变量x, 取其值并将值赋给变量b



A diagram showing a variable `b` in a light blue box containing the value `0x1234`.

◆ `*p2=0x5678;` 访问p2, 引用p2所指的对象 (也是变量x), 为其赋值, 从而x的值变为0x5678



A diagram showing two pointers, `p1` and `p2`, each in a yellow box containing `&x`. Red arrows from both boxes point to a green box labeled `x` containing the value `0x5678`.

◆ `b=*p1+2;` 访问表达式具有高优先级, 一般作为整体, 可以参与所引用变量可以参加的所有运算。



A diagram showing a variable `b` in a light blue box containing the value `0x567a`.

`*p1+2=0x5678+2=0x567a`

◆ `printf( "%x, %c" , *p1, *pc);` /*输出 p1 和 pc 所指对象的值
即输出结果为: 567a, 9 */

2、深入理解间访表达式:

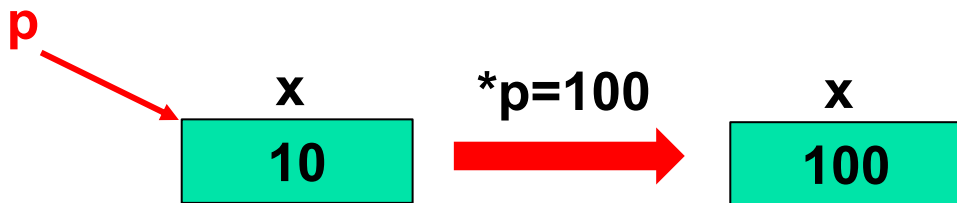
设有 **T x, *p=&x;** (这里 **T** 指某种类型, 具体如 int、char 等)

(1) 任何时候, 对于**间访表达式 *p**的解释都是指 **p 所指的对象**, 而不是 p 自己。

如: **int x=10, y, *p=&x;**

y=*p; 等价于 **y=x**, 即**取 p 所指对象 x 的值**给 y 赋值。

***p=100;** 等价于 **x=100**, **为 p 所指对象 x 赋值**, 使 x 的值被修改为100。 (而不是p的值修改为100)

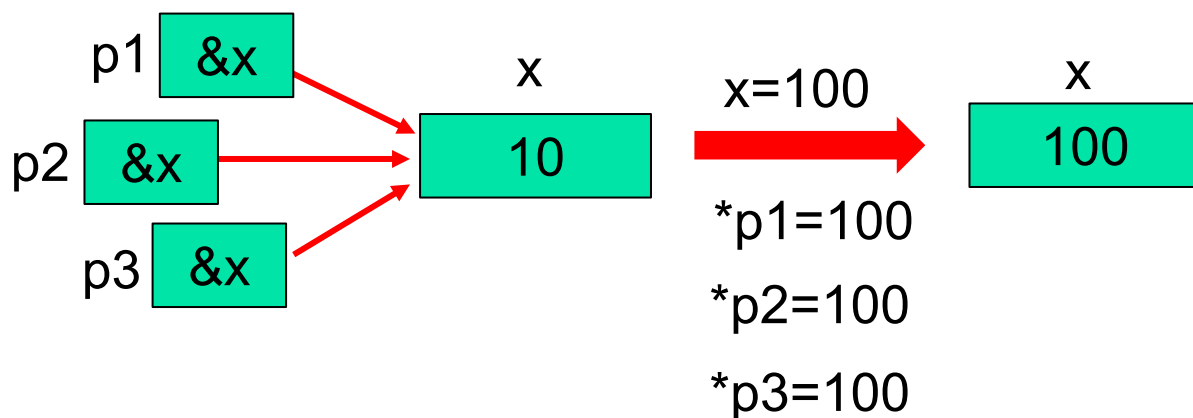


(2) 当有**多个同类型**的指针变量具有相同的值时，如：

```
int x=10, *p1=&x;
```

```
int *p2=&x, *p3=p1;
```

它们都指向同一对象，访问其中任何一个访问到的都是同一个对象：它们指向的共同对象，如这里的整型变量x。



以下是**等价的表达式**：

`x=100`

`*p1=100`

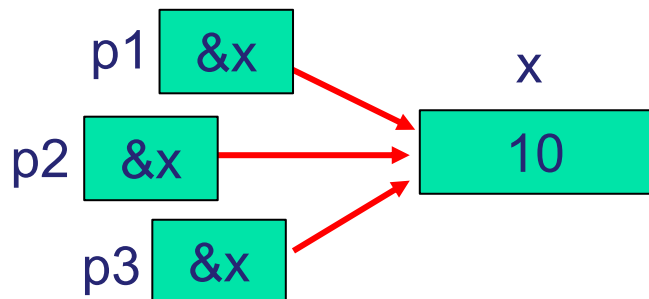
`*p2=100`

`*p3=100`

- ◆ **3个指针变量是相互独立的**；但因它们具有相同对象的地址，所以**访问它们得到的对象是同一个**。

注：只要指针变量的**类型相同**且具有**相同的值**，则访问它们到的都是同一对象。

如：



`y=x;`

`y=*p1;`

`y=*p2;`

`y=*p3;`

都是取 x 的值100赋给 y。

在指针做函数参数时，认识到这点非常重要。

(3) 间访表达式 “*** 指针**” 中，**指针**不一定是指针变量，但**必须为地址型的量**，不能是其它任何非地址类型的值或表达式。

如： `int x=3, y[5]={1, 2, 3, 4, 5}, *p = &x;`

`*p = 6;` ✓

`*x = 20;` ✗

注：这里 `p+1` 不是一个正确的地址，但语法没问题。

`*(p+1) = 7;` ✓

`*(y+x) = 10;` ✓

`*(x+y) = 100;` ✓

“指针” 部分可以是合法的**地址运算表达式**。

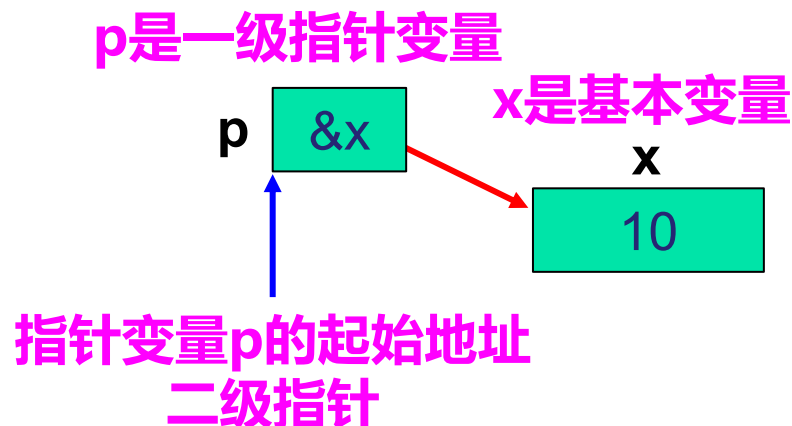
(详见8.1.4指针的算术运算)

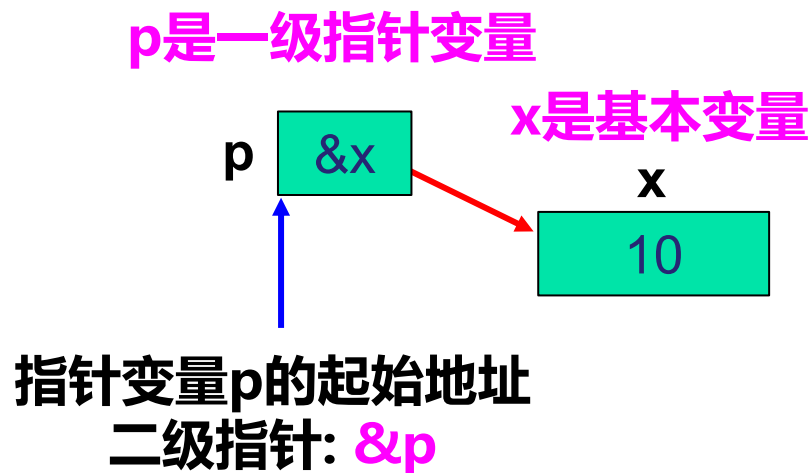
6、指针变量的地址和多级指针

如 `int x=10, *p=&x;`

这里，将简单变量（`x`）称为**基本变量**，将**基本变量的地址**称为**一级指针**，将**指向基本变量的指针变量**称为**一级指针变量**。

一个一级指针变量也有它自己的存储空间（4字节），从而也有它自己的起始地址。这个地址称为一级指针变量的地址，亦即一级指针变量的指针，称为二级指针。





- ◆ 可以通过 **&一级指针变量** 取得一个一级指针变量的地址：
如 $\&p$ 。
- ◆ 同时，还可以定义另一个指针变量来保存这个地址 $\&p$ ——
这个指针变量是保存**一级指针变量地址**的指针变量，称为**二级指针变量**。
- ◆ 以此类推，还有三级指针、三级指针变量

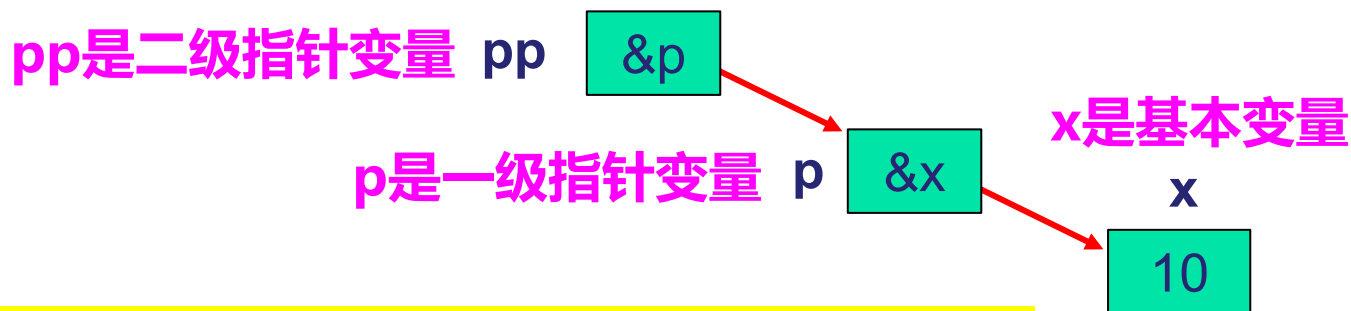
◆ 二级指针变量的声明:

一级指针变量的类型是 **T ***，故可以将二级指针变量看作是基类型为 **T *** 的指针变量，其类型为 **T ****。

声明二级指针变量的一般形式为: **T ** 二级指针变量名;**

如: `int x=10, *p=&x, **pp;`

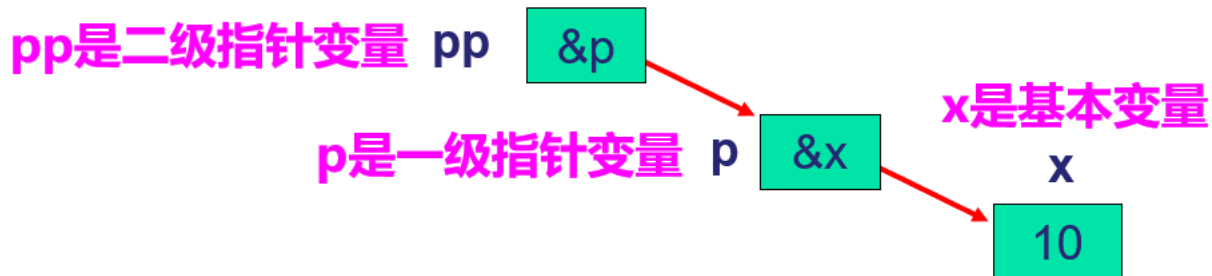
`pp=&p;`



◆ 二级指针变量指向一级指针变量

如: `int x=10, *p=&x, **pp;`

`pp=&p;`



- ◆ 直接间访二级指针变量得到的是一级指针变量:

`*pp` $\longrightarrow$ p

- ◆ 二级间访二级指针变量得到的是基本变量:

`**pp` 等价于 `*( *pp )` $\longrightarrow$ `*(p)` $\longrightarrow$ x

`y=*pp;` ✗

`y=**pp;` ✓

◆ 更多级指针变量的声明

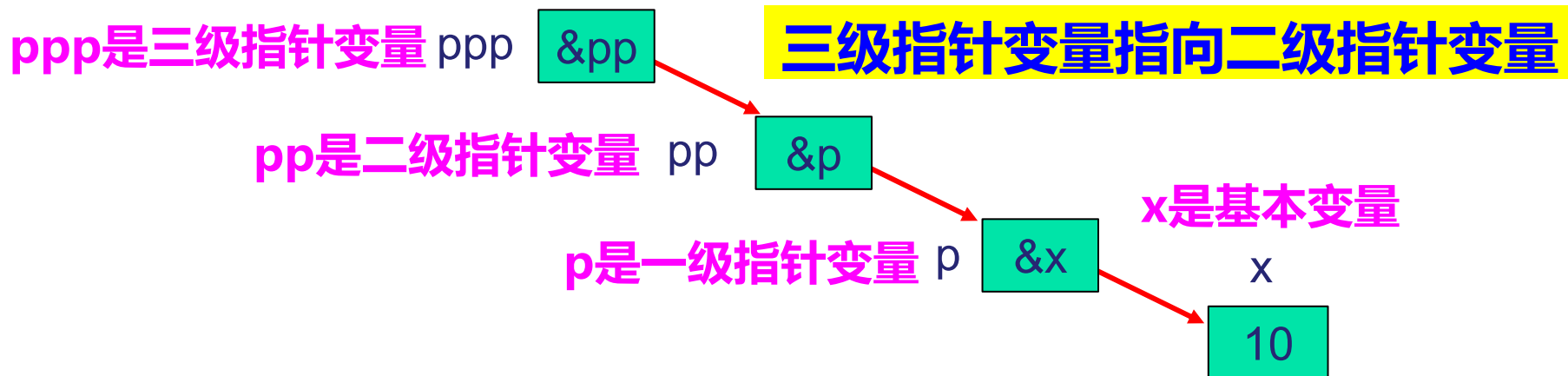
二级指针变量的地址称为**三级指针**，定义**三级指针变量**来保存二级指针变量的地址。

三级指针变量可看作是**基类型为 T^{**}** 的指针变量，其**类型为 T^{***}** 。声明三级指针变量的一般形式为：

T^{*} 三级指针变量名;**

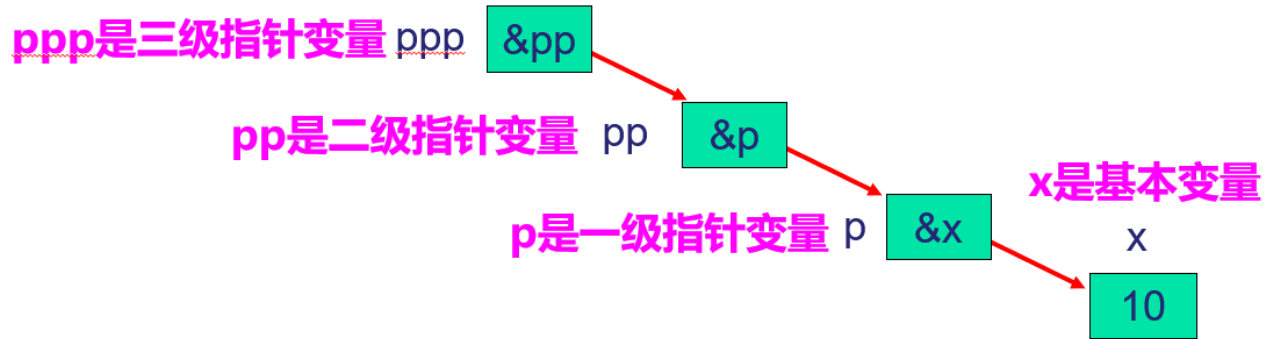
如： `int x=10, *p=&x, **pp, ***ppp;`

`pp=&p;      ppp=&pp;`



如: `int x=10, *p=&x, **pp, ***ppp;`

`pp=&p; ppp=&pp;`



- ◆ 直接间访三级指针变量得到的是二级指针变量:

`*ppp` → `pp`

- ◆ 二级间访三级指针变量得到的是一级指针变量:

`**ppp` 等价于 `*( *ppp )` → `*(pp)` → `p`

- ◆ 三级间访三级指针变量得到的是基本变量:

`***ppp` 等价于 `*( **ppp )` → `*( *pp )` → `*p` → `x`

`y=***pp;` ✓

例 多重指针之间的指向关系

```
#include "stdio.h"
```

```
int main(void)
```

```
{
```

```
    int x=12345, *p=&x, **pp=&p, ***ppp=&pp;
```

```
    printf("&x=%p\tx=%d\n", &x, x);
```

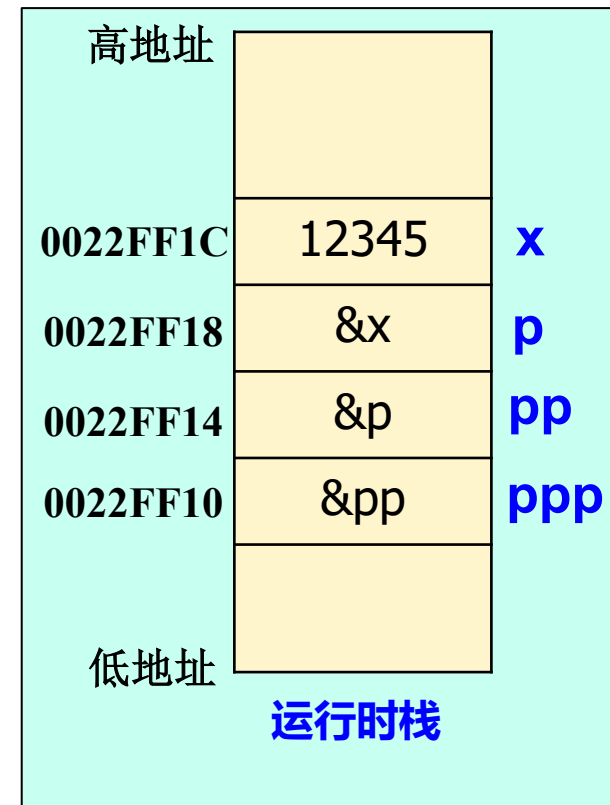
```
    printf("&p=%p\tp=%p\n", &p, p);
```

```
    printf("&pp=%p\tpp=%p\n", &pp, pp);
```

```
    printf("&ppp=%p\tppp=%p\n", &ppp, ppp);
```

```
    return 0;
```

```
}
```



输出:

&x=0022FF1C	x=12345
&p=0022FF18	p= 0022FF1C
&pp=0022FF14	pp= 0022FF18
&ppp=0022FF10	pp= 0022FF14

以此类推，有**四级指针**、**五级指针**.....，可以定义**四级指针变量**、**五级指针变量**、.....来保存各级指针。

总结：

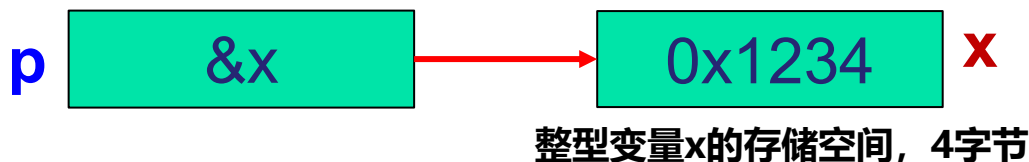
- ◆ 普通变量 x 是**基本变量**；基本变量的地址是**一级指针**；
- ◆ **一级指针变量** $T * p$ 保存基本变量的地址（一级指针），间访一级指针变量得到基本变量： **$*p$ 等价于 x** ；
- ◆ **二级指针变量** $T ** pp$ 保存一级指针变量的地址（二级指针），间访二级指针变量得到一级指针变量： **$*pp$ 等价于 p** ；
- ◆ **三级指针变量** $T *** ppp$ 保存二级指针变量的地址（三级指针），间访三级指针变量得到二级指针变量： **$*ppp$ 等价于 pp** ；
- ◆

7、再看指针的基类型

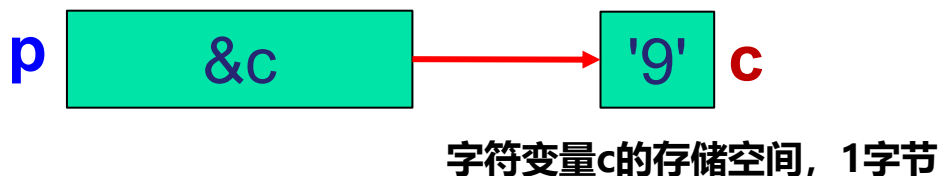
思考：对内存而言，**所有字节的地址都是一样的**，那么间访一个地址（指针变量），如 ***p**，如何知道访问的对象是什么呢？

——显然，**由其基类型决定**：

- ◆ 若 p 的基类型是 **int** 型 (**int *p**)，则 ***p** 就对应一个 **4 个字节**的整型数据；



- ◆ 若 p 的基类型是 **char** 型 (**char *p**)，则 ***p** 就对应一个 **1个字节**的字符数据；

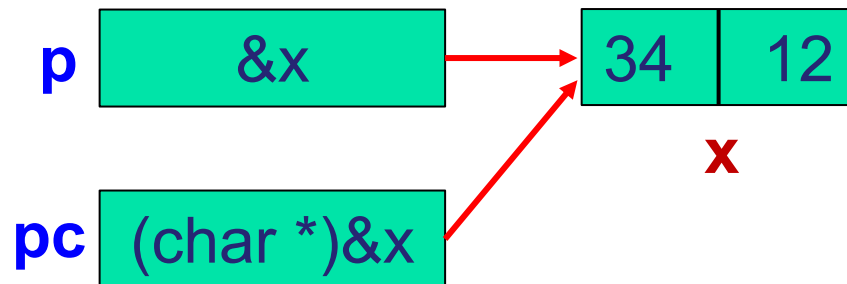


进一步思考：

```
short int x=0x1234, *p=&x;
```

```
char *pc;
```

```
pc=(char *) p;
```



若有： `printf("*p=%x, *pc=%x", *p, *pc);`

问：输出是什么？

输出： `*p=1234, *pc=34`

思考原因

核心： 根据指针变量的基类型引用所指“对象”

再有:

```
char c[6]={0x12, 0x34, (char)0xab, (char)0xcd, 0x56, 0x78};
```

若 `char c[6]={0x12, 0x34, 0xab, 0xcd, 0x56, 0x78};`

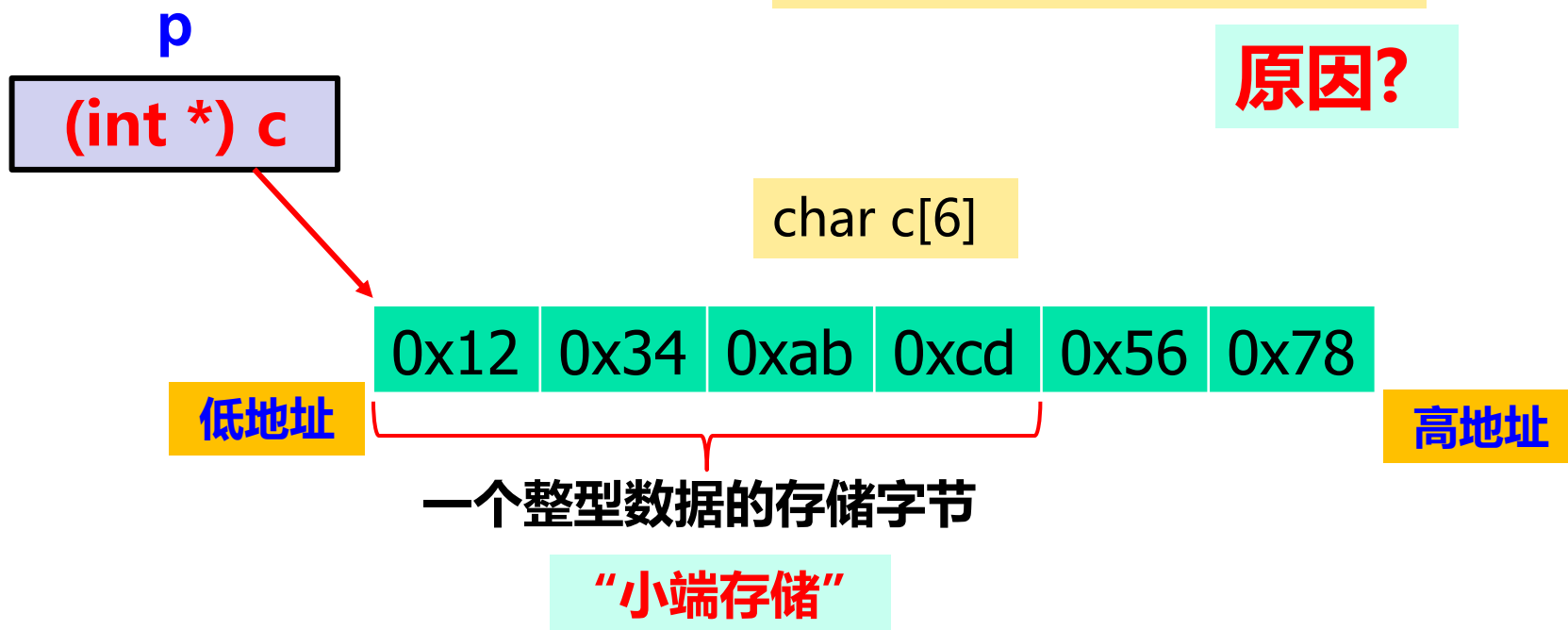
```
int *p= (int *) c;
```

```
printf("*p=%x", *p);
```

问: 输出是什么?

输出: *p2=cdab3412

原因?



8.1.4 指针的算术和关系运算

1、指针的算术运算

指针的算术运算只有**加、减**运算，不能做乘、除（运算受限）。

设有 **T *p, *q;**

int n;

即：p 和 q 是事先定义好的两个同类型的指针变量，n 是一个正整数。

且：**假设都已正确赋值。**

- ◆ 指针加法运算：只能做 **p+n**，不能做 **p+q**
- ◆ 指针减法运算：可以做 **p-n**，也可以 **p-q**

(1) 指针±整数

```
T *p, *q;  
int n;
```

◆ $p+n$ 的值等于 $|p|+n\times\text{sizeof}(T)$ ，且类型与 p 相同；

即： $p+n$ 的结果是一个值比 p 的值大 $n\times\text{sizeof}(T)$ 的
同类型地址。

◆ $p-n$ 的值等于 $|p|-n\times\text{sizeof}(T)$ ，且类型与 p 相同；

即： $p-n$ 的结果是一个值比 p 的值小 $n\times\text{sizeof}(T)$ 的
同类型字节地址。

◆ $p\pm n$ 的结果还是一个指针（地址值），且值不是在 p 的基础上简单 $\pm n$ ，而是 $\pm n\times\text{sizeof}(T)$ ，其中 T 是指针的基类型。

◆ 一般， $p\pm n$ 只有在数组空间中才有实际意义，见后。

```
T *p, *q;  
int n;
```

设有 $T\ a[N]$;

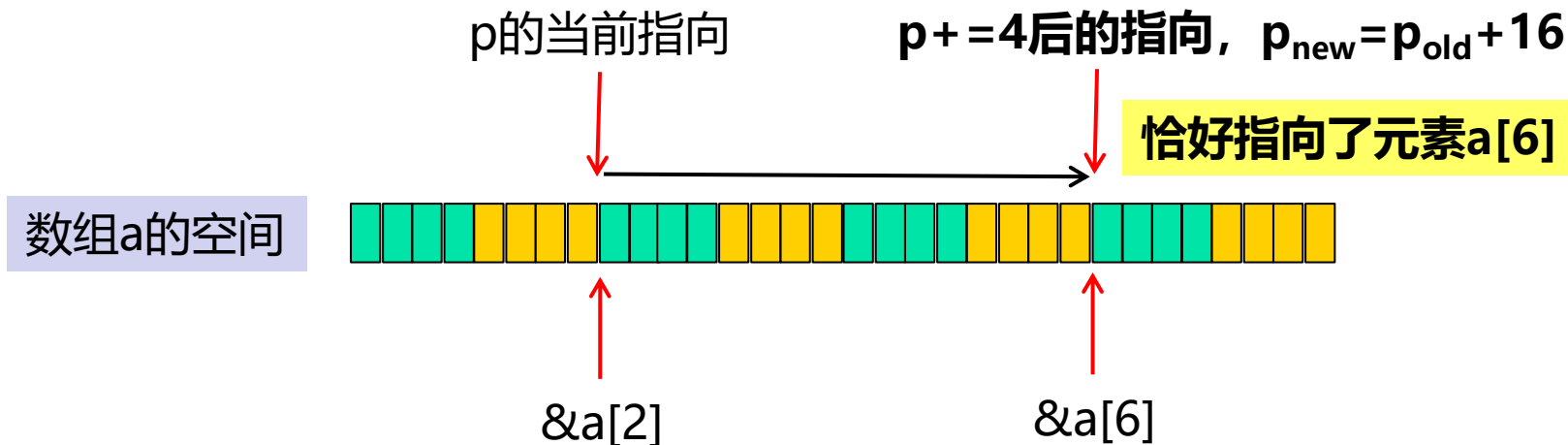
$p = \&a[i]$; 即 p 指向 **T 类型一维数组** 的某个元素 $a[i]$ 。

(a) $|p| + n \times \text{sizeof}(T)$ 是 p 当前位置后面第 n 个元素的地址。

若有: $p += n$: 将使指针 p “后移 n 的元素” 而指向 $a[i+n]$ 。

如, 设 $\text{int } a[8] = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $*p = \&a[2]$;

则: $p += 4$ $|p| = |p| + 4 * \text{sizeof}(\text{int}) = |p| + 16$

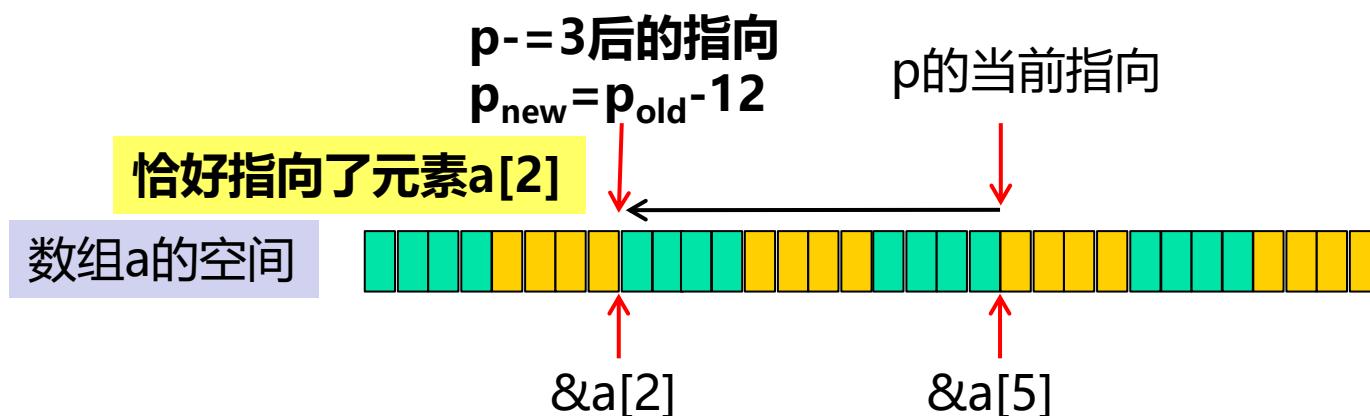


(b) $|p| - n \times \text{sizeof}(T)$ 是 p 当前位置前面第 n 个元素的地址。

若有: $p -= n$: 将使 p 指针 “前移 n 个元素” 而指向 $a[i-n]$ 。

如, 设 $\text{int } a[8] = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $*p = \&a[5]$;

则: $p -= 3$ $|p| = |p| - 3 * \text{sizeof}(\text{int}) = |p| - 12$



只有在**数组空间**中, 才可以说 $p \pm n$ 后指向**之后或之前**的其它元素; 非数组空间, 这一指向没有逻辑意义。所以一般只有在数组空间 $p \pm n$ 才有实际意义 (超出数组有效范围时的合法性另计)。

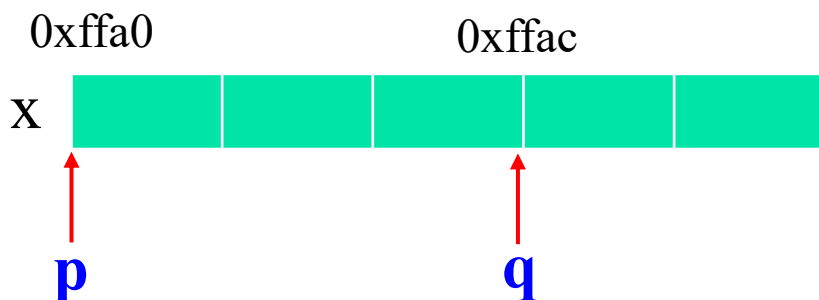
(2) 指针 - 指针

设指针变量p、q指向**同一数组**的元素，且 $p \leq q$ 。则，

q-p 等于 p 和 q 之间元素的个数

即， **$q-p = (|q|-|p|) / \text{sizeof}(T)$**

如：设 `int x[5]={1, 2, 3, 4, 5}`, `*p=&x[0]`, `*q=&x[3]`;



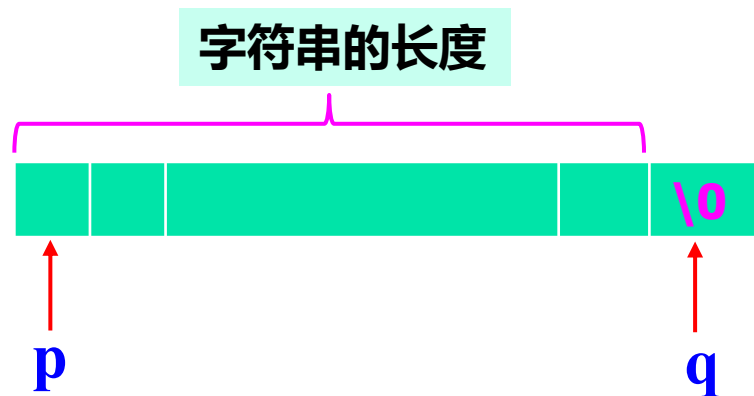
$$\begin{aligned} q-p &= (0xffac-0xffa0)/\text{sizeof}(\text{int}) \\ &= 12/4 \\ &= 3 \end{aligned}$$

即 p 到 q 之间有 3 个元素

同理，只有在数组空间，指向**同一数组**的两个指针之间相减才有实际意义。

例 利用指针变量相减求字符串的长度。

```
int strlen(char *p)
{
    char *q=p;
    while(*q) /* 查找字符串的结束位置 */
        q++;
    return (q-p); /* q-p就是字符串的长度 */
}
```



2、指针的++和--运算

设 p 是某类型的指针,

- **++p**: 相当于 **$p=p+1$** , 使指针 p 自增 “1” (实际增量是`sizeof(T)`), 指向其后面的一个元素;
- **--p**: 相当于 **$p=p-1$** , 使指针 p 自减 “1” (实际增量是`-sizeof(T)`), 指向其前面的一个元素。
- **p++**: 先使用指针 p 的当前值, 到达序列点时, p 再自增 1 并指向其后面的一个元素;
- **p--**: 先使用指针 p 的当前值, 到达序列点时, p 再自减 1 并指向其前面的一个元素。

同样, 指针的++、--运算只有在数组空间中才有实际意义。

例 设有声明: `int y, x[3]={1,3,5}, *px=x;` **px指向x[0]**

以下语句相互独立, 请分析各语句的结果:

- (1) `++*px;` 相当于: `++(*px);` 结果是使x[0]自增1, x[0]的值变为2。
- (2) `y=*++px;` 相当于: `y=*(++px);` 结果是先使px自增1而指向x[1], 然后间访得到x[1], 将x[1]赋给y, 使得y等于3。
- (3) `y=*px++;` 相当于: `y=*(px++);` 结果是先间访px, **右结合** 取得x[0]的值赋给y, 使得y等于1, 然后px自增1而使得px指向x[1]。

基于定义：int y, x[3]={1,3,5}, *px=x;

(1) ***px++** 和 **(*px)++** 是不等价的。

◆ ***px++**：++ 的作用对象是指针变量 px，最终使 px 增 1 而指向 x[1];

◆ **(*px)++**：++ 的作用对象是指针变量 px 所指的對象 x[0]，使 x[0] 的值增 1，px 的指向不变。

(3) **&x[0]**, **x** 及 **px**：三者的值相同，都是 x[0] 的地址，类型也都是 int *。

(2) ***(&x[0])**, ***x**, ***px** 及 **x[0]**：四者访问到的对象都是 x[0]。

应用举例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, a[10];
```

```
    int *p=a; /* p 指向 a 的第一个元素 a[0]*/
```

```
    for(i=0; i<10; i++)
```

```
        scanf("%d", p++);
```

P 就是指针（地址），不要再 &p

/* 利用 **p++**，每次输入后使 p 后移 1 个位置而指向 a 的下一个元素*/

```
    for(p=a, i=0; i<10; i++)
```

```
        printf("%d\n", *p++);
```

/* 相当于先 ***p**，输出当前元素，再 **++**，使 p 后移一个位置而指向下一个元素*/

```
    return 0;
```

```
}
```

3、指针的关系运算

指针之间可以进行 $<$ 、 $<=$ 、 $>$ 、 $>=$ 、 $==$ 、 $!=$ 关系运算。

设 p 和 q 是**两个 T 类型**的指针变量，且经过正确赋值。

- ◆ $p==q$ ：表示 p 和 q 的值相同，即表示**它们指向同一个对象**，

注：它们本身不是同一个指针变量，只是值相同，指向同一个对象。

- ◆ $p!=q$ ：表示 p 和 q 的值不相等，分别指向了不同的对象。

- ◆ $p>q$ ：内存中， p 所指的位置在 q 所指位置的后面（地址增大的方向）。



- ◆ $q<p$ ：内存中， q 所指的位置在 p 所指位置的前面。

应用举例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, a[10];
```

```
    int *p=a;    /* p指向a的第一个元素a[0]*/
```

```
    for(; p < a+10; p++)
```

```
        scanf("%d", p); /* p 就是指针, 不要再 &p */
```

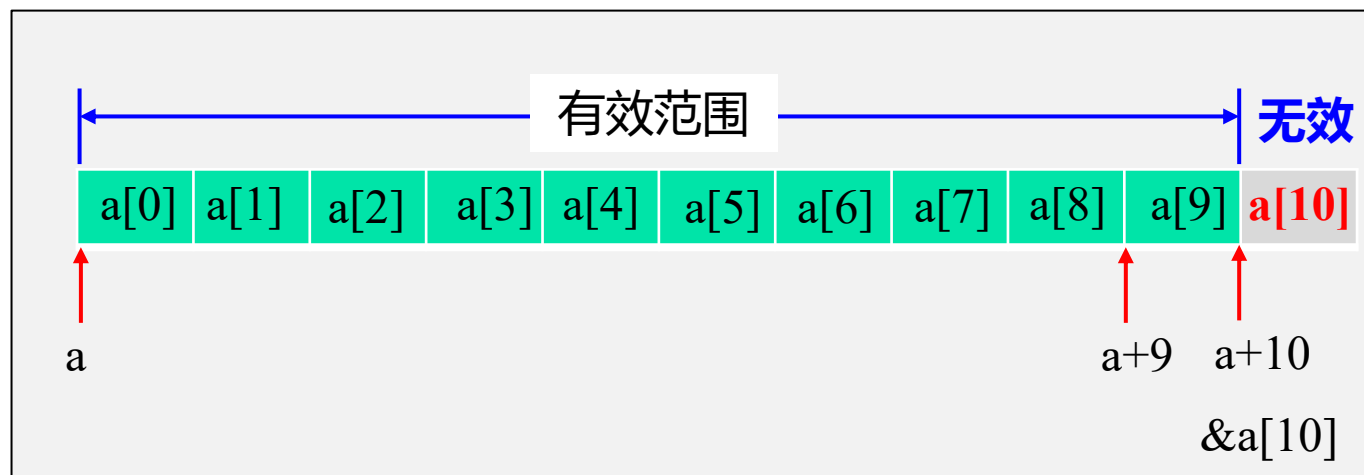
```
    for(p=a; p < &a[10]; )
```

```
        printf("%d\n", *p++); /* 先访问p, 输出当前元素*/
```

```
        /* 再由++使p后移指向下一个元素*/
```

```
    return 0;
```

```
}
```



注：如这里 **a+10**、**a[10]**，即使它们指示的位置或代表的元素超出了数组的有效范围，**但语法上是正确的表示，只是一个“超界”的位置或值**，正是利用这个性质来**控制循环在有效范围内循环**。

- 
- ◆ C语言规定**指针的关系运算只能限于同类型指针之间**。
 - ◆ **不同类型指针之间的关系运算被视为“非法”操作。**

由于不同类型的变量在内存中分配的存储不同，并且对于不同的数据类型，定义于其上的操作和运算也不相同。因此，尽管所有指针的值都是等长的二进制正整数，但从关系运算的操作语义上看，**允许不同类型的指针进行比较没有实际意义**，并且只会给语言和程序带来副作用，故不允许。

8.1.5 空指针、无类型指针、悬挂指针

1、空指针：值为 0 的指针称为**空指针**。

- ◆ C语言中，**预定义符号常量 NULL** 的值为 0，可以将 **NULL** 赋给任何类型的指针变量，**使指针变量置为 0 而成为空指针**。

T * p = NULL;

```
#define NULL 0
```

或 **p=0;** **完全等价**

- ◆ **不能间访空指针并赋值**：如上，若 ***p=5;** 则严重错误。✗

0 也是地址编码，而且对应操作系统的**中断向量区**（内存地址为0的单元是1号中断向量所在的存储单元），不能随意访问。否则会导致计算机系统崩溃。

- ◆ **空指针常用作选择或循环的控制条件**。

如：**while(p!=NULL)** 本质上就是**判断 p 的值不等于 0**。

2、无类型指针：类型为 **void *** 的指针称为**无类型指针**，或 **void 指针**。

(1) 声明 **void 指针变量**：**void * p;**

(2) void 指针是一种**准通用型指针**：C语言规定，**任何类型的指针（地址）都可以给 void 指针变量赋值**。

但由于 void 指针“无类型”，所以**不能间访 void 指针**。

如：设有声明 `int x=1, *px=&x;`

`char ch='a', *pc=&ch;`

void *vp;

则：**vp=px;** /* 合法 */

vp=pc; /* 合法 */



***vp='b';** /* 非法 */

x=*vp; /* 非法 */



- ◆ 若把 **void 指针** 赋给 **非 void 指针**，则需要进行**类型强制转化**。

如，设有 `int x=1, *px=&x;`

`char ch='a', *pc=&ch;`

`void * vp;`

`vp=px;` `/* 合法 */`

`vp=pc;` `/* 合法 */`

则： `px=(int *)vp;` `/* 合法 */`

`pc=(char *)vp;` `/* 合法 */`

- ◆ **指针的类型强制转化**就是将地址的类型强制限定为指定的类型，从而可以**按照新类型间访指针所指空间里的对象**。

空指针和 void 指针的区别

- ◆ 空指针指一个指针的值等于 0，但可以有类型；

如 `int * p=NULL;`

`double * pd=0;`

- ◆ void 指针指一个指针的类型为 “void * 类型”。

- ◆ 空指针也可以是 void 类型的空指针：`void * vp=NULL;`

- ◆ NULL 可以赋给任何类型的指针，而不用考虑类型是否一致。

如 `int * p=NULL;`

`double * pd;`

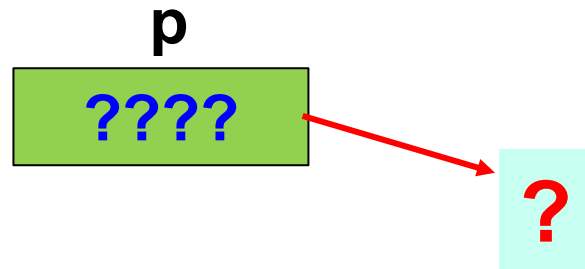
`pd=NULL;`

3、悬挂指针

如果一个指针变量声明后**没有明确赋值**，没使其具体指向一个已知的对象，则称该指针处于“**无所指**”状态。处于“无所指”状态的指针称为**悬挂指针**。

如 `int x, *p;`

`*p = 6;` ❌



◆ 使用悬挂指针很危险、会发生不可预计的后果

- ❑ 因为不知道指向哪里，所以随意引用悬挂指针一般会造成程序不能正常运行甚至系统崩溃，因此**要避免使用指针悬挂**。
- ❑ **使用指针前一定要明确赋值，使其指向确定的对象。**

8.1.6 指向常量的指针、常量指针

(1) 指向常量的指针变量

用 **const** 声明常量: **const T x;**, **x** 有存储空间, 只是值不能修改。可以定义指针变量指向它, 称为**指向常量的指针变量**。

如: **const int x1=100, x2=200, *px=&x1;**

x1、x2 是 const int型 常量

px 的指针类型是 const int *

- **指向常量的指针变量指向的对象是常量**, 不能通过指针间访问修改常量的值, 如 ***px=300;** **✗** **试图修改常量x1的值**
- **指向常量的指针变量本身是变量**, 可以修改它自己的值, 从而使其指向不同的常量 (类型必须一致)。

如 ***px=&x2;** **✓**

(2) 常量指针

初始化后，值不能再修改的指针“变量”称为**常量指针**。

- ◆ 声明**常量指针**的方式：**T * const p = 初值;**

如：**int x=100, *const cpx=&x;**

- ◆ **常量指针本身是“常量”**，其值在初始化之后不能再被修改。

如：**cpx=&x2;** ✗

- ◆ 但**常量指针指向的对象可以为常量也可以为变量**。如果指向的是变量，则可以通过间接访问常量指针修改所指对象的值。

如：***cpx=300;** ✓

思考：若有 `const int x=100, *const cpx=&x;` 会怎样？

8.2 指针参数

- 函数的参数可以是**指针型的形参**。这样的参数叫**变参**。

如： `int MyFunc( int * p )`
 {

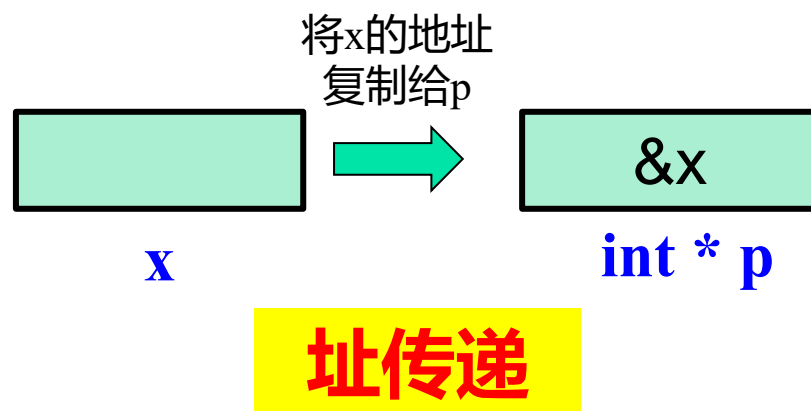
 }

- 调用**有指针参数的函数**时，需用**地址型的量作为实参**。

如： `int main(void)`
 {
 int x;

 MyFunc(&x);

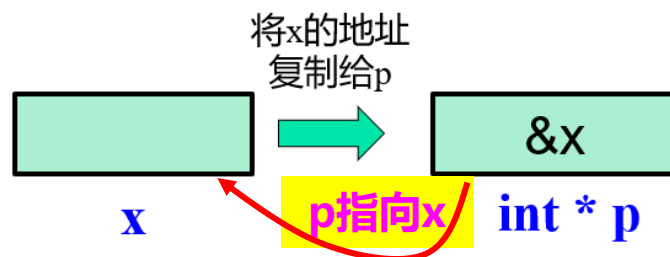
 }



址传递：

在调用含有**指针型参数**的函数时，系统首先为**指针型形参临时分配**存储单元，然后**将实参地址复制给形参指针变量**，这样**形参指针就具有和实参地址一样的值**。

——这种传送地址的参数传递方式称为“**址传递**”。



- (1) **址传递本质上还是值传递**：是将实参地址复制给形参指针；
- (2) 形参指针是独立于实参和“实参地址”而存在的存储单元，但因为具有和实参地址一样的值，所以**形参指针和“实参地址”都“指向”内存的同一对象**。

◆ 值传递和址传递对主调函数实参值的影响：

设有以下函数，观察 **swap()** 函数的行为特征。

```
void swap(int u, int v)
{
    int t;
    printf( "in function swap, before: u=%d and v=%d\n", u, v);
    t=u;
    u=v;
    v=t;    /* 交换u和v的值*/
    printf("in function swap, after: u=%d and v=%d\n", u, v);
}
```



```
#include "stdio.h"
```

```
void swap(int u,int v);      /* 交换函数, 普通的值型参数*/
```

```
int main(void)
```

```
{  int x,y;
```

```
    printf("input two integers for x and y please!\n");
```

```
    scanf("%d%d", &x, &y);
```

```
    printf("in function main, before, x=%d and y=%d\n", x, y);
```

```
    swap(x, y);    /* 直接用实参名 (值) 调用swap函数*/
```

```
    printf("in function main, after, x=%d and y=%d\n", x, y);
```

```
    return 0;
```

```
}
```



执行程序，有以下结果：

input two integers for x and y please!

2 5

in function main, **before: x=2 and y=5**

in function swap, **before: u=2 and v=5**

in function swap, **after: u=5 and v=2**

swap中u和v的
值是被交换了

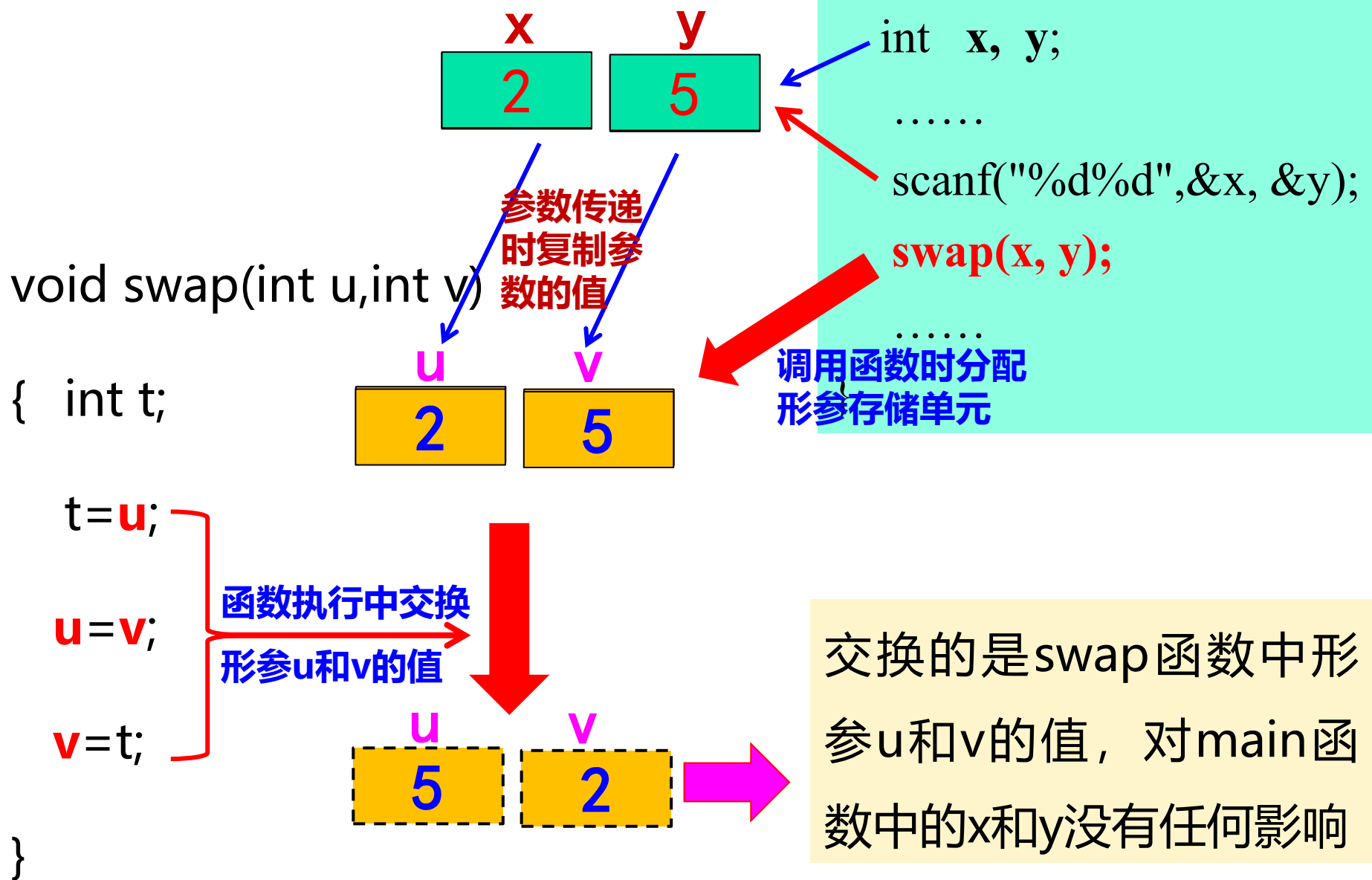
in function main, **after: x=2 and y=5**

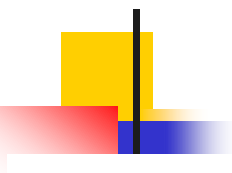


但main函数中的实参x
和y的值并没有被交换

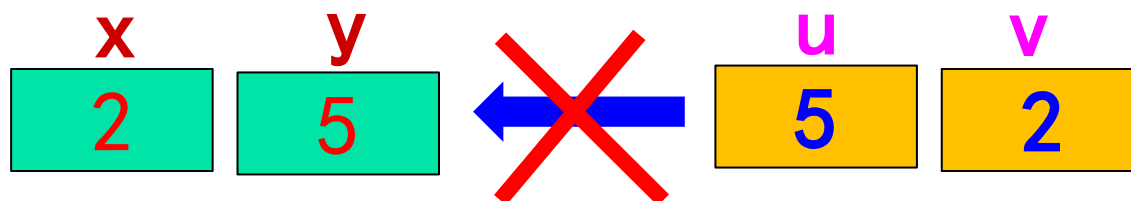
Why?

原因：值传递





形参 u 和 v 是实参 x 和 y 的一个**副本**，物理上各是不同的实体。**swap 函数中对 u 和 v 的交换对实参 x 和 y 没有任何影响。**



所以，**仅仅通过值传递，不能达到在被调用函数中修改主调函数中的实参的目的。**

◆ 怎样才能做到让被调函数“修改”主调函数的实参呢？

—— 使用指针型参数

```
void swap2(int *p1, int *p2)
```

```
{
```

参数是指针型的（通常称为**变参**）

```
    int t;
```

```
    t=*p1;
```

```
    *p1=*p2;
```

```
    *p2=t;
```

```
}
```

通过**间访**，访问指针所指对象，
进而达到“**修改实参**”的目的。


```
#include "stdio.h"
```

```
void swap2(int *p1,int *p2); /* 有指针型形参的交换函数*/
```

```
int main(void)
```

```
{
```

```
    int x,y;
```

```
    printf("input two integers for x and y please!\n");
```

```
    scanf("%d%d",&x,&y);
```

```
    printf("before: x=%d and y=%d\n",x,y);
```

```
    swap2(&x, &y);    /*调用时用 x, y 的地址作为实参*/
```

```
    printf("after: x=%d and y=%d\n",x,y);
```

```
    return 0;
```

```
}
```

执行程序，有以下结果：

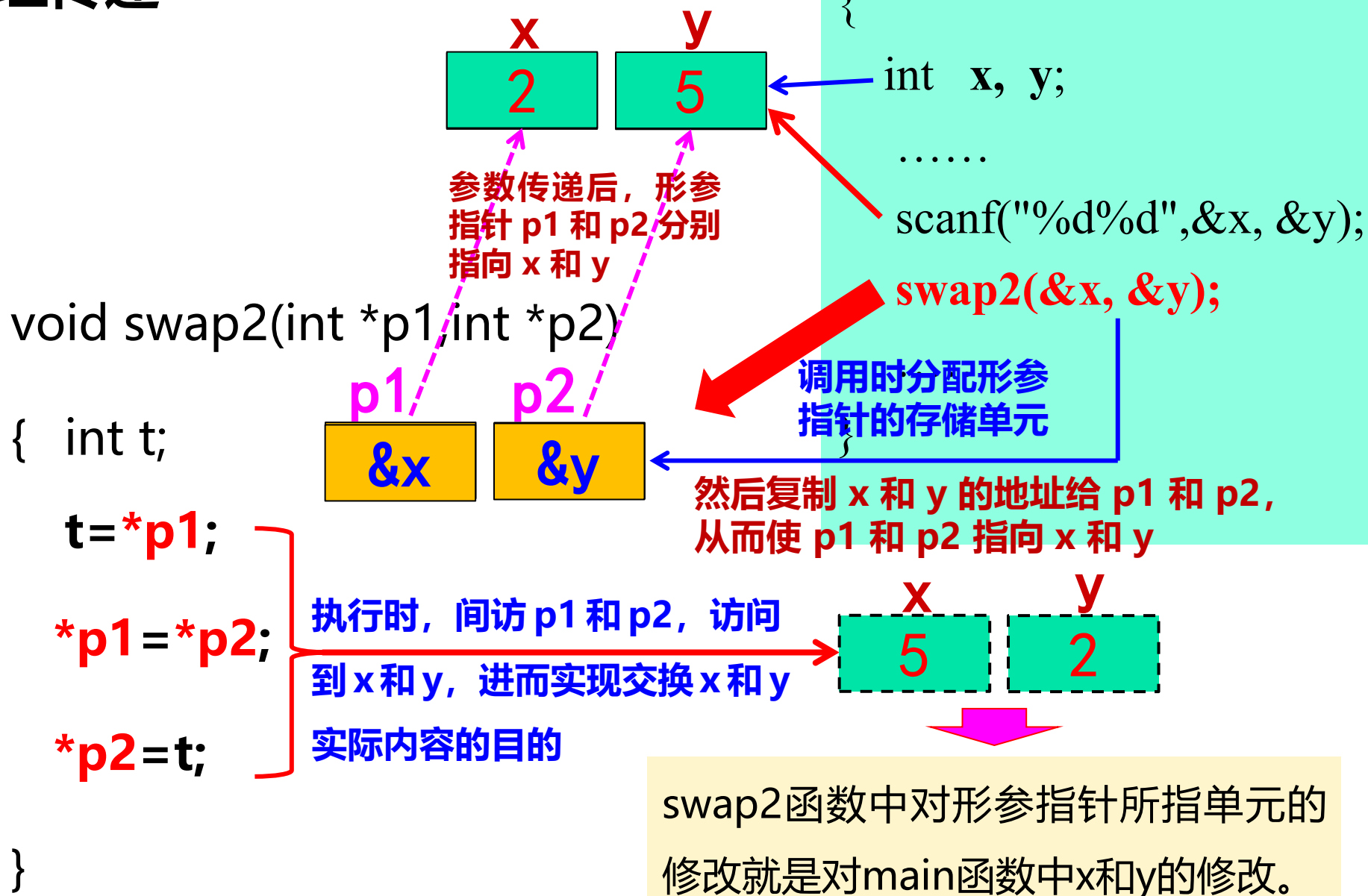
input two integers for x and y please!

2 5

in function main, before: **x=2 and y=5**

in function main, after: **x=5 and y=2**

址传递



再例：在main函数中调用 **getint** 函数，并将 **getint** 得到的数据带回。

```
#include "stdio.h"
```

```
void getints(int *x)
```

```
{
```

```
    scanf("%d", x);
```

```
}
```

```
int main(void)
```

```
{
```

```
    int x;
```

```
    printf("input x: ");
```

```
    getint(&x);
```

```
    printf("output x=%d", x);
```

```
    return 0;
```

```
}
```

这里 x 已经指针（地址）型，
不用再取x的地址。

5
main中的x

这里的 x 是 main 的局部变量，
和 getint 的形参 x 没有关系

执行程序，有以下结果：

input x: 5

output x=5

8.3 指针和一维数组

数组代表的是内存中一片连续的存储区域。

以一维数组 `int a[5];` 为例

数组元素	a[0]	a[1]	a[2]	a[3]	a[4]
元素地址	a+0	a+1	a+2	a+3	a+4
	&a[0]	&a[1]	&a[2]	&a[3]	&a[4]

- **数组名可以直接当指针使用**，代表数组的起始地址；
- 一维数组的起始地址同时也是其第一个元素的地址（二者的值和类型都一致），因此**一维数组名也是指向其第一个元素的指针**。
- 每个元素都有自己的地址，可以取地址：**&a[i]**，
 - ◆ 根据指针的算术运算，有：**&a[i] == a+i**。

定义基类型是数组元素类型的指针变量，如： **int * p;**

(1) 令 **p** 指向数组的元素，然后利用 **p** 实现对数组元素的访问。

如： `int a[10], *p = a;` /*令p指向数组的第一个元素a[0]*/

`*p=1;` /*间访p，访问p当前所指对象，即a[0]，所以等价于a[0]=1*/

`*(p+3)=1;` /*先计算相对于p当前所指位置、元素偏移为3处的地址，然后间访该地址
访问相应的数据对象，即x[3]，然后赋值，所以等价a[3]=1*/

`p[3]=1;` /*访问相对于p当前所指位置、元素偏移为3处的元素，即x[3]，
然后赋值，所以也等价于a[3]=1*/

二者等价

数组形式

(2) 将数组名直接用作指针，通过对数组名的间访来访问元素：

`*a=1;` /*相当于 a[0]=1，解释同 `*p=1` */

`*(a+3)=1;` /*相当于 a[3]=1，解释同 `*(p+3)=1` */

8.3.1 一维数组元素的指针表示

1、对数组元素引用形式 $E1[E2]$ 的解释

C语言中，对数组元素的引用表达式 $E1[E2]$ 的解释为：

$$*(E1 + E2)$$

这里，视 $E1$ 是一个起始地址、 $E2$ 是位置偏移（即下标）。

- ◆ $E1 + E2$ ：计算相对起始地址 $E1$ 、位置偏移为 $E2$ 处的地址；
- ◆ $*(E1 + E2)$ ：间访地址 $(E1 + E2)$ ，得到该处的数据元素。

因此，

- ◆ 从数组的角度， $E1[E2]$ 是访问数组 $E1$ 的下标为 $E2$ 的元素；
- ◆ 从指针的角度， $E1[E2]$ 被视为间访地址 $(E1 + E2)$ ，从而访问相对于起始地址 $E1$ 、位置偏移为 $E2$ 处的对象。

这里的E1可以是数组名，也可以是指针变量或其它合法的地址量。

(1) 当 E1 为数组名时

如，设有说明：**int x[6];**

x 即是 E1，此时为基于数组名的元素访问形式

◆ 数组名[下标]的元素表示：

x[0]、x[1]、x[2]、x[3]、x[4]、x[5]

◆ 取元素的地址：&x[0]、&x[1]、&x[2]、&x[3]、&x[4]、&x[5]

◆ 数组中各个元素的地址 基于数组名+偏移 的指针表示：

x+0, x+1, x+2, x+3, x+4, x+5

◆ 通过间访上述地址访问数组的元素：

*(x+0), *(x+1), *(x+2), *(x+3), *(x+4), *(x+5)

(2) 当 E1 是一个指针变量时

声明基类型与数组类型相同的指针变量，令其指向数组中的元素，然后基于指针变量对数组元素进行访问。

如： `int x[6], *p=x;` 或者 `p=&x[0];` p 指向数组 x 的开始位置

元素的地址：

`&x[0]:` `(p+0)` 或 `(x+0)`

`&x[1]:` `(p+1)` 或 `(x+1)`

.....

`&x[i]:` `(p+i)` 或 `(x+i)`

引用元素：

`x[0]:` `p[0]`、`*(p+0)`、`*(x+0)`

`x[1]:` `p[1]`、`*(p+1)`、`*(x+1)`

.....

`x[i]:` `p[i]`、`*(p+i)`、`*(x+i)`

需要注意的是：

(I) **数组名是地址常量**，不能被修改。但 **p** 是变量，可以被修改

如： `int a[6], *p=a;`

`a=a+1;` **×**

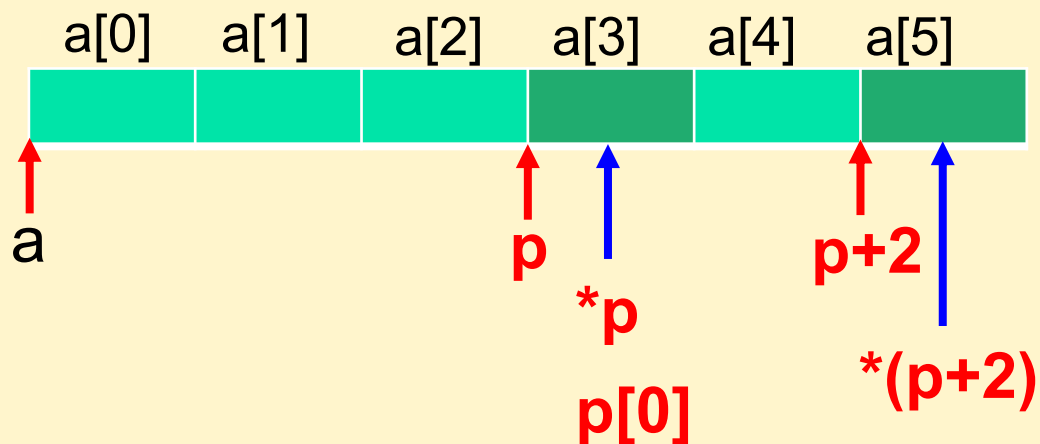
`p=p+1;` 或 `p++;` **✓**

任何时刻：

- ◆ **a** 始终都是数组的起始地址，指向数组的第一个元素；
- ◆ ***a** 或 **a[0]** 是数组的第一个元素；
- ◆ **a+i** 是数组中元素 **a[i]** 的地址，**a+i==&a[i]**，***(a+i)==a[i]**。

(II) 指针变量的值可以修改，赋予不同元素的地址值，就指向不同的元素：

- ◆ 如：若 $p = \&a[3]$ ，则赋值后 p 等于 $a+3$ ； p 指向 $a[3]$ 元素。
- ◆ $*p$ 或 $p[0]$ 仅代表访问当前所指元素（即相对于 p 当前所指位置、偏移为 0 处的元素）；
- ◆ 而， $p+i$ ：是相对 p 当前所指位置、偏移为 i 处的地址，
 $*(p+i)$ （或 $p[i]$ ）：是该处的元素。



如： $p = \&a[3];$



例: int **x**[8]={1, 2, 3, **4**, 5, **6**, **7**, 8}, ***p = &x[3];**

p指向数组的第四个元素

int * q;

p[0]=100; /* **相当于 x[3]=100** */

q = &p[2]; /* **p[2] 相当于 x[5], &p[2] 相当于 &x[5], 所以结果是 q 指向 x[5]** */

p = &q[1]; /* **q[1] 相当于 x[6], 之后 p 指向 x[6]** */

printf("%d", *p); /* **输出的是元素 x[6] 的值, 即 7** */

例，设有声明：**char s[20]="abcdef", *pc=s;**

对于数组名 s 和指针 pc，以下哪些是合法操作？

设 i 是合法定义的整型变量

➤ s+i

➤ *(s+i)

➤ ++*s

➤ (*s)++

合法操作

➤ s=pc

➤ &s

➤ *++s

➤ *s++

非法操作

➤ pc=s

➤ pc+i

➤ ***(pc+=i)**

➤ *++pc

➤ ++*pc

➤ *pc++

➤ **(*pc)++**

➤ **&pc**

合法操作

++和*都是单目运算符，同时出现时严格按照运算符的**优先级（2级）**和**结合性（右结合）**来解析

8.3.2 一维数组参数的指针表示

- ◆ 函数的形参可以是一维数组

```
void bubble(int a[ ], int n)
{
    int i;
    for(i=0;i<n;i++)
        a[i]=a[i]* a[i];
}
```

- ◆ 调用时，用实参数组名
作为实参：

```
int main( )
{
    int x[3]={1,2,3};
    bubble(x, 3);
    return 0;
}
```

- ◆ 数组型形参不是真正的数组，本质上就是一个指针型形参。
- ◆ 调用时，采用地址传递，将实参数组的地址复制给形参数组名。

因此，**形参数组**和**形参指针**在形式上可以互换：

形式1：形参指针 & 实参数组

◆ 将函数的形参声明为指针：

```
void bubble(int *a, int n)
{
    int i;
    for(i=0;i<n;i++)
        a[i]=a[i] * *(a+i);
}
```

也有不同但等价的元素引用形式

◆ 调用时，仍用**实参数组名**作为实参

```
int main( )
{
    int x[3]={1,2,3};
    bubble(x, 3);
    return 0;
}
```

◆ 址传递

◆ 形参数组和形参指针在使用形式上通用。

形式2: 形参指针 & 实参数组元素的地址

◆ 将函数的形参声明为指针:

```
void bubble(int *a, int n)
{
    int i;
    for(i=0;i<n;i++)
        a[i]=a[i] * *(a+i);
}
```

◆ 调用时, 用数组元素的地址 作为实参

```
int main()
{
    int x[3]={1,2,3};
    bubble(&x[0], 3);
    return 0;
}
```

思考: 若调用为:

bubble(&x[1], 2);

将是什么含义?

形式3: 形式数组 & 实参数组元素的地址

- ◆ 将函数的形参声明为形式数组:

```
void bubble(int a[ ], int n)
{
    int i;
    for(i=0;i<n;i++)
        a[i]=a[i] * *(a+i);
}
```

- ◆ 调用时, 用数组元素的地址作为实参

```
int main( )
{
    int x[3]={1,2,3};
    bubble(&x[0], 3);
    return 0;
}
```


◆ 可以直接对形式数组名进行指针运算

形参数组名本质上是“**指针变量**”，其值可以改变。

```
void bubble(int a[ ], int n)
```

```
{
```

```
    int i;
```

```
    for(i=0;i<n;i++)
```

```
        *a = *a * a[0];
```

***a 和 a[0] 都是访问 a 当前所指元素**

```
        a++;
```

改变a的值

```
    }
```

- ◆ 普通数组名是地址常量，不可修改；
- ◆ 但**形式数组名**仅是“**指针变量**”，可以进行算术运算而变化；

例 用指针变量表示一维数组参数，计算两个向量的内积

$$\vec{A} \cdot \vec{B} = \sum_{i=0}^{n-1} a_i b_i$$

```
double scalar(double *a, double *b, int n)
```

```
{
```

```
    double s = 0;
```

```
    int i;
```

```
    for(i=0; i<n; i++)
```

```
        s+=((*a++)*(*b++));
```

```
    return s;
```

```
}
```

基于指针运算扫描数组的元素



```
#define SIZE 10
```

```
int main(void)
```

```
{
```

```
double x[SIZE], y[SIZE], *px, *py;
```

```
for(px=x; px<x+SIZE; px++)    /* x+SIZE是元素x[SIZE]的地址 */
```

```
    scanf("%lf", px);
```

```
    getchar();                /*读取换行符*/
```

```
    for(py=y; py<&y[SIZE]; )    /* &y[SIZE]就是y+SIZE/
```

```
        scanf("%lf", py++);
```

```
    printf("the scalar of vector x and y is %lf\n", scalar(x, y, SIZE));
```

```
    return 0;
```

```
}
```

计算两个double型数组内的元素**对应相乘**

*8.3.3 高精度计算：超长整数加法运算(自学)

在C语言中，16位机中的无符号整型变量能够存储的最大整数是65535，超过了这个范围就必须用长整型long int。32位无符号长整型变量能够存储的最大整数是 4294967295l。

更大的数据如何表示和处理呢？

如： **12345678901234567890+98765432109876543210**

- ◆ 这里只考虑两个正大整数的加，且最大位数一样。

1. 读入一个大整数

用整型数组保存大整数，数组中每个元素代表一个整数的一位。

```
void getBigNum( int *x, int lim)
{
    int i, t, c;
    int *p1, *p2;
    for(i=1; i<=lim; i++) *(x+i)=0;           /* 数组清零 */
    for(i=1; i<lim && isdigit(c=getchar()); i++)
        *(x+i) = c-'0';                      /* 数字字符-'0' 转换为对应的十进制数值 */
    *x = i-1; /* 用x[0]保存输入的整数的位数 */
    for(p1=x+1, p2=x+i-1; p1<p2; p1++, p2--) /* 反转, 使得低位数在低下标处 */
        t=*p1, *p1=*p2, *p2=t;
}
```

反转：开始读入的大整数在数组中的表示顺序是高位在低下标处，低位在高下标处。为后续处理方便，需要将数组中的内容反转，使数字顺序变为高位在高下标处，低位在低下标处。

如，设int x[16], lim=15;

输入：'1', '2', '3', '4', '5', '6', '7', '8', '9', '0', '1', '2'

初始数组中的内容：

(对应的大整数是：123456789012)

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]	x[8]	x[9]	x[10]	x[11]	x[12]	x[13]	x[14]	x[15]
1	2	3	4	5	6	7	8	9	0	1	2	0	0	0	0
高位											低位				

反转后的内容：

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]	x[8]	x[9]	x[10]	x[11]	x[12]	x[13]	x[14]	x[15]
2	1	0	9	8	7	6	5	4	3	2	1	0	0	0	0
低位											高位				

循环的方向

2. 基于数组，实现两个大整数的加

反转后，**个位在数组的下标0处**，所以从 $i=0$ 起逐位相加即可。

但需要考虑以下问题：

- (1) **进位**：个位相加，可能向十位进1，其余位相加要考虑低位进过来的1，还可能向更高位进1（包括最高位）。
 - ◆ 用整型变量**carry**记录进位，开始时**carry=0**；
 - ◆ 设当前两位是**xi**和**yi**，先计算 **$xi+yi+carry$** ，若和大于10，置**carry=1（向高位进1）**，否则置**carry=0**；
- (2) **两个整数的位数不一样怎么办？**

由于数组中超出最高位的元素都置为了0，所以以最“长”的大整数为参照，算到“最高位”即可。

```
void addBigNum( int *z, int *x, int *y)
```

```
{
```

```
    int i, carry=0, n;
```

```
    for(i=1;i<=N;i++) *(z+i)=0;    /*先将结果数组清零 */
```

```
    n=max(*x, *y); /* n为两个加数中较长的位数 */
```

```
    for(i=1; i<=n; i++) {
```

```
        *(z+i) = *(x+i) + *(y+i) + carry;    /* 带进位的逐位相加 */
```

```
        carry=*(z+i)/10; /* 计算新进位 */
```

```
        *(z+i) %= 10; /* 计算本位 */
```

```
    }
```

```
    if(carry) { /*处理最后一位的进位*/
```

```
        *(z+i) = carry;
```

```
        i++; /* 最高位有进位，结果的位数要+1 */
```

```
    }
```

```
    *z = i-1; /* 用z[0]保存结果的位数 */
```

```
}
```


3. 输出大整数

因为整数的高位在大下标处，所以要将整数数组的内容从后往前依次输出。

```
void putBigNum( int *x)
{
    int *p, n;
    n=*x;    /* 整数的位数 */
    for(p=x+n; p>x; p--) /* 从高位开始输出 */
        printf("%d", *p);
}
```

4. 总控过程

```
#include <stdio.h>
#include <ctype.h>
#define N 200 /*最大位数*/
#define max(a,b) ((a)>(b) ? (a): (b))

void getBigNum( int *, int);
void addBigNum( int *, int *, int *);
void putBigNum( int *);

int main()
{
    int x[N+1], y[N+1], z[N+2];
    printf("输入被加数: "); getBigNum(x, N);
    printf("输入加数: ");   getBigNum(y, N);
    addBigNum(z, x, y);    /* 计算z=x+y */
    putBigNum(z);
    putchar('\n');
    return 0;
}
```

如：

$$123456789012 + 87654321098 = 211111110110$$

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]	x[8]	x[9]	x[10]	x[11]	x[12]	x[13]	x[14]	x[15]
2	1	0	9	8	7	6	5	4	3	2	1	0	0	0	0

+

y[0]	y[1]	y[2]	y[3]	y[4]	y[5]	y[6]	y[7]	y[8]	y[9]	y[10]	y[11]	y[12]	y[13]	y[14]	y[15]
8	9	0	1	2	3	4	5	6	7	8	0	0	0	0	0

→ 进1
→ 进1

→ 进1 → 进1 → 进1 → 进1 → 进1 → 进1 → 进1 → 进1

||

z[0]	z[1]	z[2]	z[3]	z[4]	z[5]	z[6]	z[7]	z[8]	z[9]	z[10]	z[11]	z[12]	z[13]	z[14]	z[15]
0	1	1	0	1	1	1	1	1	1	1	2	0	0	0	0

8.4 字符指针和字符串

1、声明一个**字符数组**，用字符串初始化；

然后**声明一个字符指针变量**，使其**指向字符数组**。

如： `char astr[ ]= "It is a string", *pstr = astr;`

此时可视为指针 **pstr 指向了 astr 中字符串的第一个字符**，可以利用 pstr 访问 astr 中的字符串或字符串中的字符，如：

<code>printf("%s\n", pstr);</code>	<code>/* 等价于 printf("%s\n", astr); */</code>
<code>printf("%c\n", *pstr);</code>	<code>/* 访问 pstr 当前所指元素，相当于 printf("%s\n", astr[0]); */</code>
<code>printf("%c\n", *(pstr+1));</code>	<code>/* 相当于 printf("%s\n", astr[1]); */</code>

- ◆ 指针变量可以通过 $\pm n$ 改变指向，以此来扫描字符串中的字符。

如： `for(pstr = astr; pstr < astr + strlen(astr); )`

`printf("%c\n", *pstr++);`

- ◆ 利用指针变量修改字符数组或字符串中的字符。

如： `char astr[ ]= "It is a string";`

`char * pstr = astr;`

`*pstr='p';` `/* 等价于 pstr[0]='p' 或 astr[0]='p' ); */`

`*(++pstr)='s';` `/* 等价于 pstr[1]='s' 或 astr[1]='s' ); */`

`scanf("%s\n", pstr);`

2、可以直接令字符指针变量指向一个**常量字符串**。

如： `char *pstr = "It is a string";`

这样的字符指针可以看作是**指向常量的指针**。

之后利用字符指针变量就可访问该常量字符串或其中的字符：

```
printf("%s\n", pstr); /* 输出 pstr 所指字符串的内容*/
```

```
printf("%c\n", *pstr); /* 输出 pstr 指向的字符串的第一个字符，即字母 I*/
```

```
printf("%c\n", *(++pstr)); /* 输出字母 t, pstr 是指针变量，可被修改*/
```

```
for(; *pstr; ) /* 利用指针运算扫描常量字符串*/
```

```
printf("%c\n", *pstr++);
```

但是： `pstr` 所指对象是**字符串常量**，只能读，不能修改其内容

```
*pstr = 'p';
```

×

```
*(pstr+1) = 's';
```

×

```
scanf("%s", pstr);
```

×

3、字符指针作为函数的参数

回顾：一维字符数组作为函数的参数（见7.5.3）

如：

```
void strcpy(char t[ ], char s[ ])  
{  
    int j=0;  
    while(t[j]=s[j++]) ;  
}
```

```
#include <stdio.h>  
void main(void)  
{  
    char str1[30];  
    char str2[ ]= "there is a boat on the lake.";  
    strcpy(str1,str2);  
    puts(str1);    /*输出字符串*/  
}
```

根据前面讲述的一维数组参数的指针表示（见8.3.2），这里同理就有：可用**字符指针参数**替换**一维字符数组参数**。

如：void strcpy(**char *t, char *s**)

```
{  
    while(*t++ = *s++);  
}
```

利用**指针运算**实现字符串复制：

- ◆ **++** 实现字符指针的移动，从而扫描整个字符串。

调用：仍然可用**实参数组名**作为**实参**调用（**址传递**）

```
void main(void)  
{  
    char str1[30];  
    char str2[ ]= "there is a boat on the lake."  
    strcpy(str1, str2);  
    .....  
}
```


也可以用**指针变量做实参**进行调用

```
void main(void)
{
    char str1[30], *ps1=str1;
    char str2[ ]= "there is a boat"
                " on the lake.";
    strcpy(ps1, str2);
    .....
}
```

都是址传递

或**元素的地址做实参**进行调用

```
void main(void)
{
    char str1[30], *ps1=str1;
    char str2[ ]= "there is a boat"
                " on the lake.";
    strcpy(&str1[0], str2);
    .....
}
```

关于字符指针和字符串的其它相关内容，参见教材P237 8.4节

8.5 指针数组

以**指针（变量）**为元素的数组称为**指针数组**。

8.5.1 指针数组的声明及使用

1、指针数组声明的基本形式：

类型名 * 标识符[常量表达式] = {初值表};

即：将数组类型说明为“指针类型”

例：**int * p[3];**

含义：声明了一个有3个元素的**整型指针数组p**，
p的每个元素都是一个 int * 的指针变量。

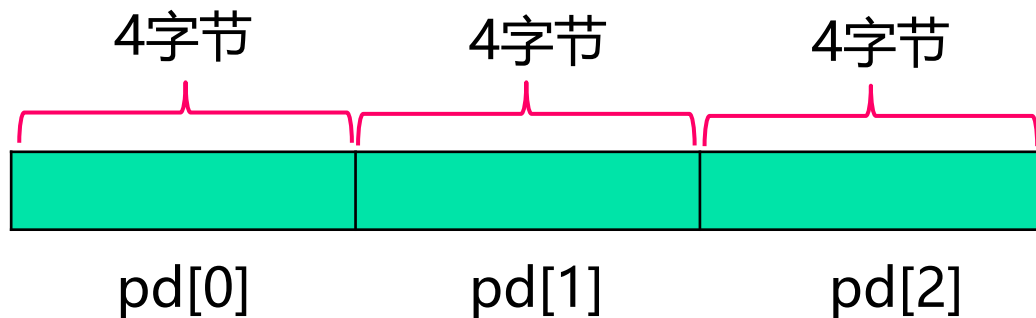
2、指针数组的存储

设在32位系统中有： $T * p[N];$

(和一般数组类似) 在内存中连续分配 **4N 个字节**的空间，并划分为 **N 段**，每段 (**4个字节**) 存储一个指针元素。

指针变量 (指针数组的元素) 都是 4 字节大小

如： $\text{double} *pd[3];$



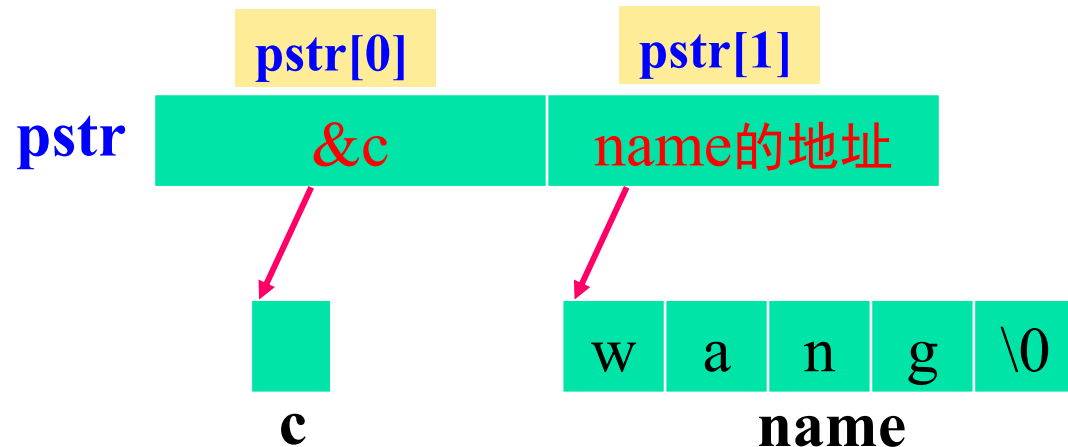
3. 指针数组的使用

(1) 指针数组中的元素都是**指针变量**，可以用**基类型**与数组元素（指针）的基类型一样的地址为指针数组中的元素赋值，从而令**每个元素都“指向”一个相应的对象**。

如：char c, name[]="wang", *pstr[2];

pstr[0]=&c;

pstr[1]= name;



赋值后 pstr 的每个元素都指向一个字符型地址。这些地址的来源可以不同，但只要**类型一致**即可。

(2) **指针数组的元素是指针变量**，可以参与同类型指针允许的所有操作。

例，设有：`int a[ ]={1,2,3}, b[ ]={4,5,6}, *p[2];`

`p[0]=a;` /* 赋值，使 `p[0]` 指向 `a` 数组的第一个元素 */

`p[1]=b;` /* 赋值，使 `p[1]` 指向 `b` 数组的第一个元素 */

`*p[0]=12;` /* 等价于 `*(p[0])`，间访 `p[0]`，得到元素 `a[0]`，然后为 `a[0]` 赋新值 12 */

`++*p[1]` /* 等价于 `++(*p[1])`，首先间访 `p[1]` 得到 `b[0]`，然后对 `b[0]` 自增1，使 `b[0]` 变为5，注：`p[1]` 没变，仍指向 `b[0]` */

`*p[1]++` /* 等价于 `*(p[1]++)`，先间访 `p[1]` 得到 `b[0]`，引用完 `b[0]` 后，`p[1]` 自增 1 而指向 `b[1]` */

注：**下标运算符[]**具有最高优先级，所以对数组元素引用时，任何时候都把 **数组名[下标]** 当作一个整体来看待。

3、用指针数组构造一种“特殊”的二维结构

设有 T^* 类型的指针数组： $T^* \text{ pa}[N];$

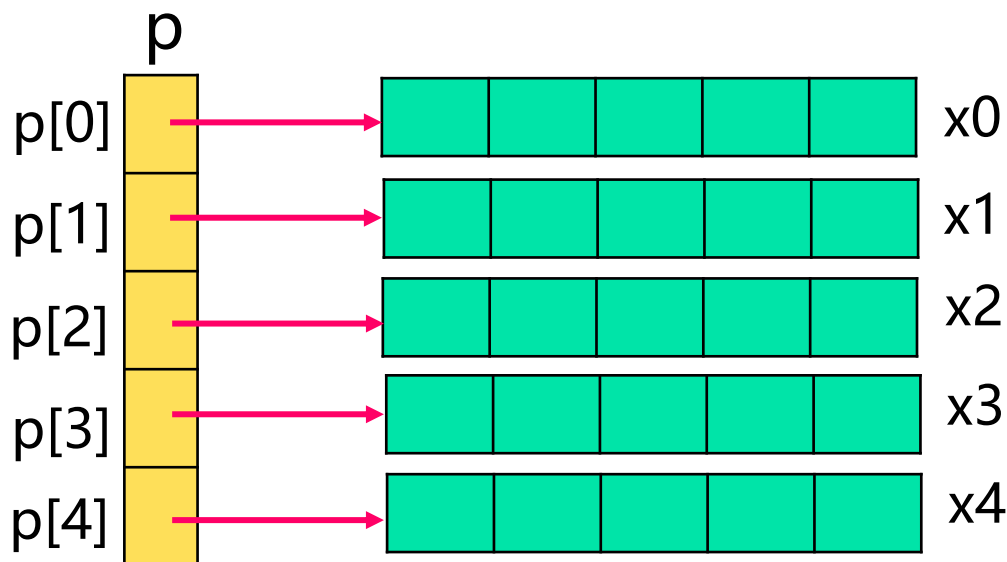
由于 pa 的每个元素都是 T^* 类型的指针变量，所以可令 pa 中的每个元素都指向一个 T 类型的一维数组，从而构成一种不同于二维数组的“特殊”的二维结构。

如, 设有: `int x0[5], x1[5], x2[5], x3[5], x4[5];`

`int *p[5],`

令: `p[0]=x0; p[1]=x1; p[2]=x2; p[3]=x3; p[4]=x4;`

形成以下二维结构



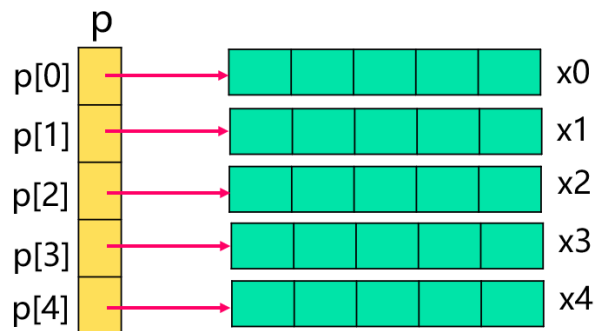
注: 这里 5 个一维数组
`x0~x4` 的空间并不一定
连续。

◆ 基于“特殊”二维结构访问其中一维数组中的元素

由于: $\mathbf{x_i[j] == *(xi+j)}$

而: $\mathbf{p[i] == xi}$

所以: $\mathbf{x_i[j] == *(p[i]+j) == p[i][j]}$



如同访问普通二维数组的方式访问这种“特殊的二维数组”

如: $x0[0]=p[0][0]$ 、 $x0[1]=p[0][1]$ 、 $x0[2]=p[0][2]$ 、.....

$x1[0]=p[1][0]$ 、 $x1[1]=p[1][1]$ 、 $x1[2]=p[1][2]$ 、.....

深入理解 $p[i][j]$:

根据 **[] 运算的左结合性**, C语言对 $p[i][j]$ 的解释就是 $\mathbf{p[i][j] == (p[i])[j]}$,

根据一维数组元素的指针表示有: $\mathbf{(p[i])[j] == *(p[i]+j)}$, 而 $\mathbf{x_i[j] == *(p[i]+j)}$,

所以 $\mathbf{x_i[j] == p[i][j]}$ 。(后面还会详述)


```
#include "stdio.h"
```

```
int main(void)
```

```
{  int x0[1], x1[2], x2[3], x3[4], x4[5];    /*这些一维数组的大小不同*/
```

```
    int *p[5], i, j;
```

```
    p[0]=x0; p[1]=x1; p[2]=x2; p[3]=x3; p[4]=x4; /*令p的元素分别指向一个一维数组*/
```

```
    for(i=0;i<5;i++)
```

```
        for(j=0; j<=i; j++)
```

```
            p[i][j]=j;
```

```
    for(i=0; i<5; i++){
```

```
        for(j=0; j<=i; j++)
```

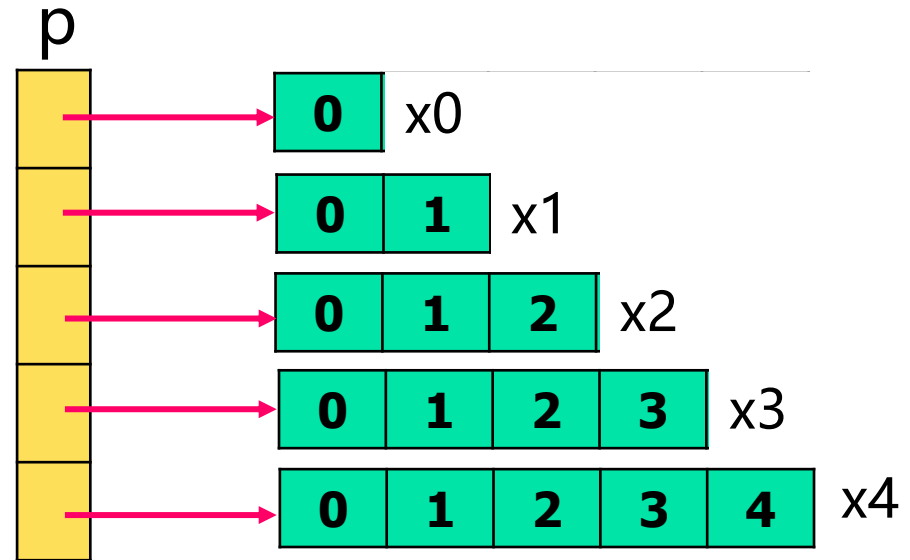
```
            printf("%4d", p[i][j]);
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```



字符指针数组的应用

(1) 令字符指针数组的元素指向字符数组。

设有：

```
char s1[20] = "can't open file!";  
char s2[15] = "read error!";  
char s3[ ] = "write error!";  
char s4[ ] = "illegal access exception!";  
char *ps[4] = {s1, s2, s3, s4};
```

使用：

```
for(i=0; i<4; i++)  
    printf("%s", ps[i]);
```

引用到字符数组

```
for(i=0; i<4; i++)  
    for(j=0; j<strlen(ps[i]); j++) {  
        printf("%c", ps[i][j]);  
        printf("\n");  
    }
```

引用到字符数组的元素

(2) 用字符指针数组构造 “字符串数组”

如: **char *perr[4];**

perr[0] = "can't open file!";

perr[1] = "read error!";

perr[2] = "write error!";

perr[3] = "illegal access exception!";

用**常量字符串**为字符
指针数组的元素赋值

或: **char *ps[]={**

"can't open file!",

"read error!",

"write error!",

"illegal access exception!"

用**常量字符串**作初值
初始化字符指针数组

};

应用举例：根据错误编号输出出错提示信息

```
void error_prompt(int err_code)
{
    char *perror[ ]={
        "can't open file!",
        "read error!",
        "write error!",
        "illegal access exception!"
    };
    printf("error code:%d :%s\n",
        err_code, perror[err_code]);
}
```

出错提示信息的字符串数组

```
#include "stdio.h "
void error_prompt(int err_code);
int main(void)
{
    int n;
    while(1){
        scanf("%d",&n);
        if(n == -1) break;
        error_prompt(n);
    }
    return 0;
}
```

输入：2

输出：error code:2 : write error!

注意以下两种定义方式都定义了“**字符串数组**”，但有区别。

```
char *ps[ ]={  
    "can't open file!",  
    "read error!",  
    "write error!",  
    "illegal access exception!"  
};
```

```
char pa2[ ][30] = {  
    "can't open file!",  
    "read error!",  
    "write error!",  
    "illegal access exception!"  
};
```

ps 是字符指针数组，其元素指向字符串常量，不能基于这些字符指针数组元素修改所指的字符串常量内容。

如：**strcpy(ps[1], "can't read!");**



pa2 是一个二维字符数组，是变量空间，常量字符串的内容复制到该空间中，每行的字符串都是可以修改的。

如：**strcpy(pa2[1], "can't read!");**

这里 **pa2[i]** 是二维字符数组 i 行的首址针。



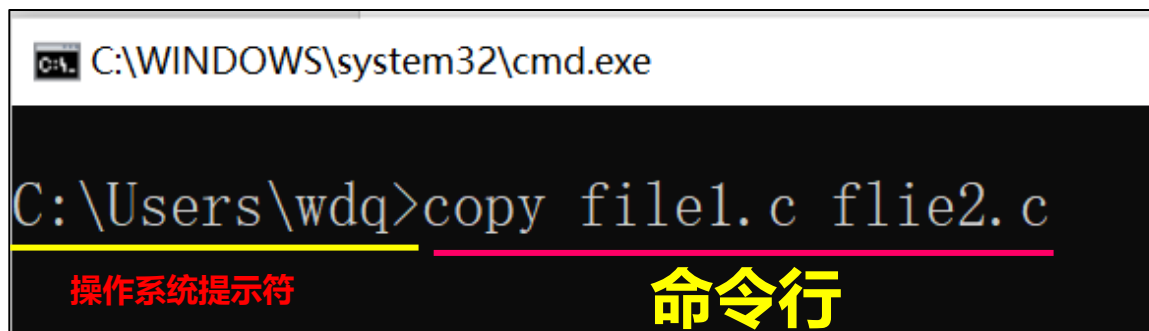
*8.6 带参数的main函数

8.6.1 命令行

在**控制台环境**中输入**可执行程序的文件名和必要的参数**执行程序的方式称为**命令行执行**。

其中，所书写的**可执行文件名及其参数**称为**命令行**。

如：



The image shows a Windows command prompt window. The title bar at the top reads "C:\WINDOWS\system32\cmd.exe". The main area of the window has a black background with white text. It shows the command prompt "C:\Users\wdq>" followed by the command "copy file1.c flie2.c". A yellow horizontal line is drawn under the command "copy file1.c flie2.c". Below this line, the text "操作系统提示符" (Operating System Prompt) is written in red, and "命令行" (Command Line) is written in yellow.

```
C:\WINDOWS\system32\cmd.exe  
  
C:\Users\wdq>copy file1.c flie2.c
```

操作系统提示符 命令行

- **带参数的可执行程序**要用**有带参main函数的程序**实现。

8.6.2 带参main函数的声明及使用

1、声明带参数的main函数的一般形式：

类型 **main**(**int argc**, **char *argv[]**)

程序执行时，操作系统会向main函数传递两个参数：

- ◆ **argc**：整型，等于**命令行中字符串的个数**；
- ◆ **argv**：是一个包含**argc**个字符指针的**字符指针数组**，其中每个元素指向命令行一个字符串。

2、系统对命令行的解析：

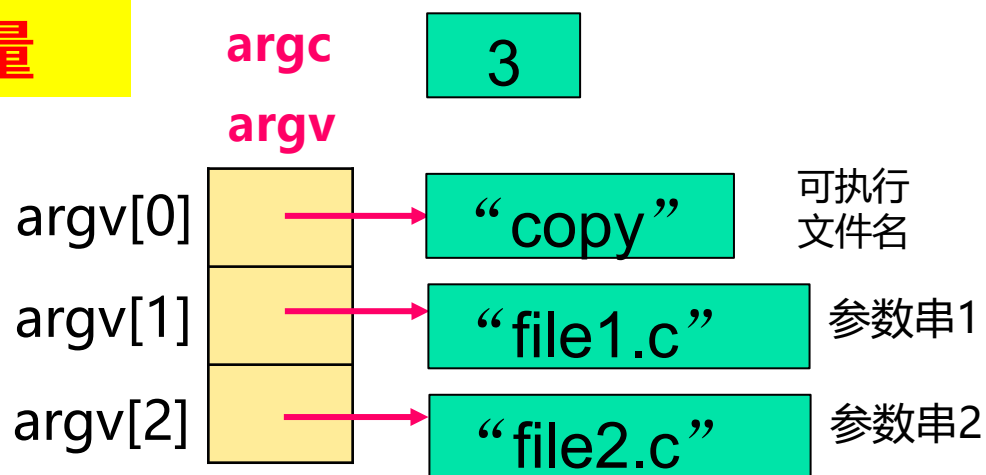
命令行中**可执行文件名及参数之间**都以**空格分隔**，系统将空格隔开的连续字符序列处理成一个个字符串（包括可执行文件名）。

执行时，系统将包括可执行文件名在内的**字符串的个数**传递给参数**argc**；同时将包括可执行文件名在内的**字符串的地址**依次赋给字符指针数组**argv**的各个元素。其中，**argv[0]指向可执行文件名字符串**，其余元素依次指向后面的参数字符串。

注：这些字符串都是字符串常量

如：

```
C:\WINDOWS\system32\cmd.exe
C:\Users\wdq>copy file1.c flie2.c
```



例8.16 显示命令行参数。

```
#include "stdio.h"
```

```
int main(int argc, char * argv[ ])
```

```
{
```

```
    int i = 0;
```

```
    for(i = 0; i < argc; i++){
```

```
        printf("argv[%d]: %s\n", i, argv[i]); /*输出指针数组元素指向的字符串*/
```

```
    }
```

```
    return 0;
```

```
}
```

假设该程序编译成的可执行文件为 **show**

命令行: **show I like C programming.**

程序的输出:

argv[0]: show

argv[1]: I

argv[2]: like

argv[3]: C

argv[4]: programming.

思考： **命令行参数都是字符串**，如何向程序输入**非字符类型**的数据，

比如整数？ —— **手工转换**

例：以命令行的方式向程序输入整数、字符和浮点数。

```
#include "stdio.h"
```

```
#include "math.h"
```

```
int main(int num, char * pstr[ ])
```

```
{
```

参数名可以是任意合法标识符

```
    char c;
```

```
    int i;
```

```
    float f;
```

```
    c= pstr[1][0];    /*第二个参数字符串的第一个字符*/
```

```
    i=atoi(pstr[2]); /*atoi 函数将一个整数字串转换成整数*/
```

```
    f=atof(pstr[3]);  /*atof 函数将一个浮点数字串转换成浮点数*/
```

```
    printf("%c\n%d\n%f\n", c, i, f);
```

```
    return 0;
```

```
}
```

设有执行： .

```
C:\WINDOWS\system32\cmd.exe
C:\Users\wdq>myprog a 1234 56.25
```

```
pstr[0]="myprog"
pstr[1]="a"
pstr[2]="1234"
pstr[3]="56.25"
```

输出：

a

1234

56.250000

8.7 指针函数

如果一个函数的**返回值**是指针（地址），则该函数称为**指针函数**。

- ◆ 声明指针函数原型的一般形式是：

类型名 * 函数名（形参表）；

- ◆ 定义指针函数的一般形式是：

类型名 * 函数名（形参表）

{

... //函数体

return 指针;

}

通过 return 语句返回一个指针（地址）

如: **char *** mystrcpy(char * t, char * s)

```
{  
    char * p = t;  
    while(*t++=*s++);  
    return p;    /*返回目的串的首址*/  
}
```

```
int main(void)
```

```
{  
    char str1[30], str2[ ] = "C language";  
    char *ptr;  
    ptr = mystrcpy(str1, str2);  
    printf("str1: %s\n", ptr);  
    return 0;  
}
```

◆ 指针函数返回的地址必须对外部有效

不能是**仅内部有效**的地址：

如： `char * getstr(void)`

```
{  
    char p[ ] = "Hello world!";  
    return p;    局部数组  
}
```

```
int main(void)  
{  
    char *str;  
    str = getstr();  
    puts(str);  
    return 0;  
}
```

◆ 编译器报错： [Warning] address of local variable 'p' returned.

改正： `char * getstr(void)`

```
{  
    static char p[ ] = "Hello world!";  
    return p;  
}
```

静态数组

相关讨论见教材P252 8.7.2

例8.18 查找子串的指针函数应用举例（自学）。

回顾：第七章用字符数组实现的strStr

```
int strStr(char s[ ], char t[ ])
```

```
{
```

```
    int j=0, k;
```

```
    for( ; s[j]!='\0'; j++) /* j扫描s串，对j指示的每个位置，测试后面连续的若干字符是不是t中的字符*/
```

```
        if(s[j]==t[0] ){ /* 先确定s的当前字符s[j]是不是t的首字符t[0]，是则代表t可能会出现*/
```

```
            k=1; /* k是计数器，同时用k扫描t串*/
```

```
            while(s[j+k]==t[k] && t[k]!='\0') /*在t串还没结束前，匹配s和t对应位置上的字符是否相等*/
```

```
                k++; /* 如果相等则k加1，然后继续匹配下一个位置*/
```

```
            if( k==strLen(t) ) /* 若连续匹配成功的字符数与串t长度相同，则表示成功*/
```

```
                return j; /*匹配成功，返回t在s中的起始下标（j 的值）*/
```

```
        }
```

```
    return -1;
```

```
}
```

用下标扫描数组

返回找到位置的下标

用指针的方式实现:

用字符指针扫描数组

```
#include "stdio.h"
```

```
char *strstr(char *s, char *t) /*s为源串,t为子串*/
```

```
{
```

```
    char *ps=s,*pt,*pc;
```

/*ps指向源串*/

```
    while(*ps!='\0'){
```

/*ps依次指向源串中每一个字符*/

```
        for(pt=t,pc=ps; *pt!='\0' && *pt==*pc; pt++,pc++); /*pc指向源串,pt指向子串;*/
```

```
        if(*pt=='\0') return ps;
```

/*到子串尾,说明匹配成功,返回子串出现位置*/

```
        ps++;
```

/*ps指向源串中的下一个字符*/

```
    }
```

```
    return 0;
```

/* 0 == NULL */

```
}
```

```
int main(void)
```

```
{
```

```
    char s1[]="abcdefghijk",s2[]="fgh",*p; /*定义源串,子串,临时指针p*/
```

```
    p=strstr(s1,s2);
```

/*在"abcde**fg**hijk"中查找"fg**h**",子串出现位置赋给p*/

```
    printf("%p\t%s\n",&s1[0],s1);
```

/*输出源串起始地址和源串*/

```
    printf("%p\t%s\n",p,p);
```

/*输出子串出现位置及从该位置起的源串*/

```
    return 0;
```

```
}
```

找到后返回主串中子串出现处的地址 (ps的值)

```
char *strstr(char *s, char *t) /*s为源串,t为子串*/
```

```
{
```

```
    char *ps=s,*pt,*pc;                                /*ps指向源串*/
```

```
    while(*ps!='\0'){                                    /*ps依次指向源串中每一个字符*/
```

```
        for(pt=t,pc=ps; *pt!='\0' && *pt==*pc; pt++,pc++); /*pc指向源串,pt指向子串;*/
```

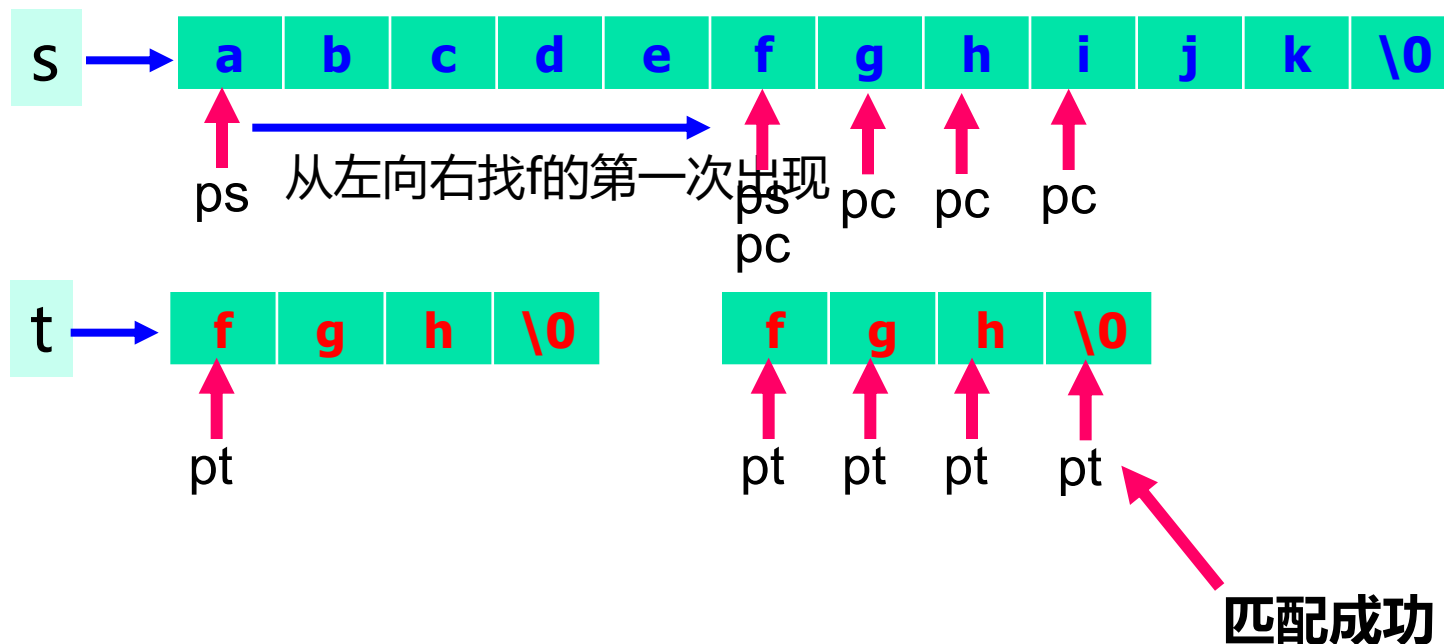
```
        if(*pt=='\0') return ps;                        /*到子串尾,说明匹配成功,返回子串出现位置*/
```

```
        ps++;                                             /*ps指向源串中的下一个字符*/
```

```
    }
```

```
    return 0;                                           /* 0 == NULL */
```

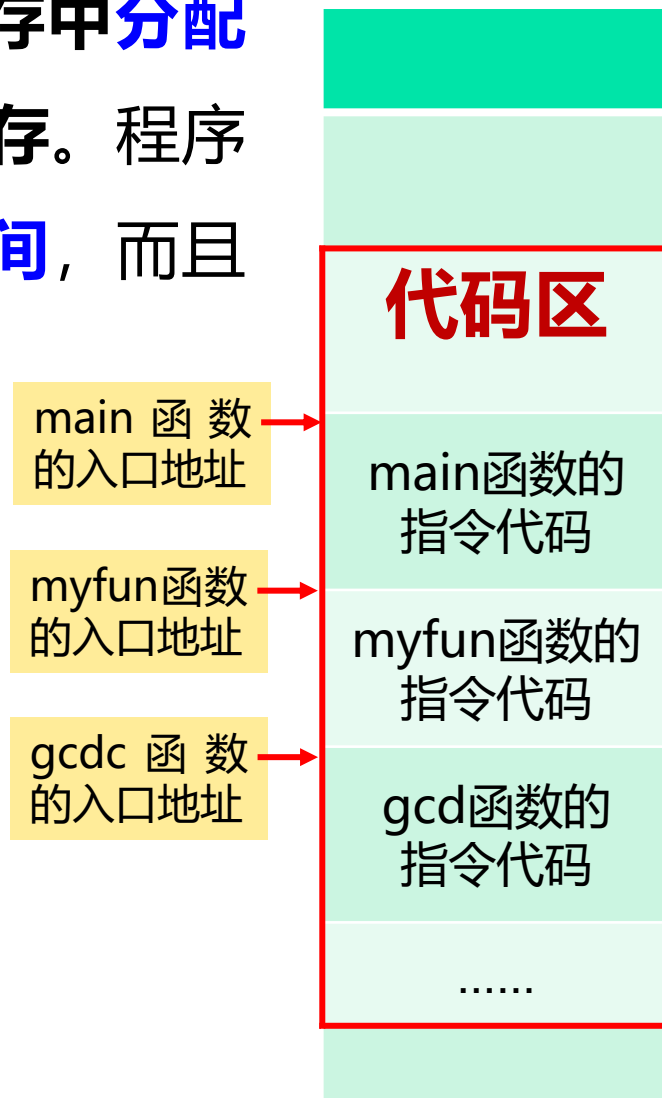
```
}
```



*8.8 指向函数的指针

可执行程序是二进制代码，是机器指令的序列，实体也是0-1串。程序执行时，系统在内存中**分配存储空间**，然后将可执行程序调入这段内存。程序中的**每条指令都有自己的若干字节存储空间**，而且是一段连续的字节空间。

每个函数的指令序列是在从第一条指令开始用一片**连续的字节空间**中顺序存放的。**函数的第一条指令是函数的入口**，该指令所占存储空间的**第一个字节的地址称为函数的入口地址**。简称函数的地址。



函数指针： 函数的第一条指令在内存中的地址是函数的入口地址。
函数的入口地址即是**函数的指针**。

C语言规定：

一个函数的函数名就代表该函数的入口地址

因此，**函数名是一种指针**，**其值是函数的入口地址值**，**所指对象就是函数**。

指向函数的指针简称为**函数指针**。

函数指针变量： 可以定义一种指针变量来保存函数的入口地址，
这种变量称为**指向函数的指针变量**，简称为**函数指针变量**。

8.8.1 指向函数的指针变量的声明

- 声明指向函数的指针变量的一般形式是：

类型 (* 函数指针变量名) (形参表);

其中，

- ◆ **(* 函数指针变量名)** 表示括号内的标识符是一个指针变量。
- ◆ **类型**表示函数指针所指函数的返回类型;
- ◆ **形参表**表示函数指针所指函数的形式参数的类型与个数。

如： **int (*fptr)(int x, int y, char * pc);**

含义：声明了一个指向函数的指针变量 **fptr**，该指针指向的函数必须是：**返回值为 int 型，参数表有三个参数，依次为 int、int 和 char * 型**（参数的数量、类型和顺序不得变化）。

1、为函数指针变量赋值

可以用**函数名**直接为函数指针变量赋值：

如， 设有函数定义：

```
void f(int x)
{
    ...
}
```

函数指针变量声明：

```
void (*pf) (int x);
```

用函数名 f 为指针变量 pf 赋值：

```
pf = f;
```

```
void (*pff) (int x, int y);
```

```
pff=f;
```



类型不匹配

说明：

- ◆ 赋值后，就称**函数指针变量指向了函数**。
- ◆ 被指函数的**返回值类型、参数的类型、个数和顺序**必须与函数指针变量声明时所指定的待指函数的返回值类型、参数类型、个数和顺序**完全一致**。否则类型不匹配，不能为指针变量赋值。

2、函数指针变量的使用

成功赋值后，函数指针变量将指向一个函数。之后可以：

(1) 通过函数指针调用它所指的函数。

有两种形式：

- ◆ **间访调用形式**：(*函数指针名)(实参表);

如：(*pf)(5);

- ◆ **直接调用形式**：函数指针名(实参表);

如：pf(5);

用函数指针调用函数和用函数名调用函数所发生的动作是相同的

(2) 函数的参数可以是函数指针型的。

方式：将函数的参数说明为**函数指针变量型**，调用时用**要调用的函数名**作为实参。

如，设有：

```
int ff (int x, void (*pf)(int))
{
    return pf(x);
}
```

```
int f1(int x)
{
    return x*x;
}
```

```
int f2(int x)
{
    return x+x;
}
```

```
int main( )
{
    int y1, y2;
    y1=ff(5, f1);
    y2=ff(5, f2);
    return;
}
```

特性：

- ◆ 可以传入不同的**函数**实现不同的功能。

用函数指针参数调用的函数称为回调函数，如这里的f1、f2。

例8.20 有全班30名学生的成绩，需要输出60分以下的成绩和大于等于90分的成绩 —— **回调函数的设计与使用。**

```
#include <stdio.h>
```

```
#define N 30
```

```
int GreaterOrEqual(int, int); /* 比较两个参数：前一个参数是否不小于后一个参数*/
```

```
int Less(int, int); /* 比较两个参数：前一个参数是否小于后一个参数*/
```

```
void findNumbers(int *, int, int, int (*)(int, int));
```

在这里传入要回调的函数

```
int main(void)
{
    int i, score[N];
    printf("输入%d个成绩: ", N);
    for(i=0;i<N;i++)
        scanf("%d", score + i);    /*数组名+偏移换算元素地址*/
    printf("低于60分的有: \n");
    findNumbers(score, N, 60, Less);
    printf("高于或等于90分的有: \n");
    findNumbers(score, N, 90, GreaterOrEqual);
    return 0;
}
```

给findNumbers传入不同的函数，从而在其内部实现不同的功能



/*大于等于被比较数compNumber的回调函数*/

int **GreaterOrEqual** (int number, int compNumber)

```
{  
  
    return number >= compNumber;  
  
}
```

/*小于被比较数compNumber的回调函数*/

int **Less** (int number, int compNumber)

```
{  
  
    return number < compNumber;  
  
}
```



/* 查找函数：在x中查找满足指定条件的数，compNumber是被比较数，**p是函数指针**，它所指向的函数用于将数组中的数与compNumber比较*/

```
void findNumbers (int *x, int n, int compNumber, int (*p)(int, int))  
{  
    int i;  
    for(i=0; i<n; i++)  
        if ( p(x[i], compNumber) )  
            printf("%d\n", x[i]);  
}
```

在这里回调函数p

3、扩展理解：从“函数类型”的角度理解函数指针。

对于函数头：

类型名 **函数名** (参数列表)

去掉函数名，将剩余部分看成是一种特定的“函数类型”定义：

类型名 (参数列表)

整体看作是一种类型声明：类型表达式

则对函数指针类型的理解可以这样：

- ◆ 指向函数的指针就是基类型是 “**类型名** (参数列表)” 的指针；而该指针的类型是 “**类型名** (*) (参数列表)”；

↑
类型表达式

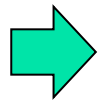
*：放在基类型的中间，并用圆括号括起来，用以说明这是一种特定的指针类型。

◆ **对函数指针变量定义的理解**可以这样：

定义函数指针变量就是声明一个类型为 **类型名 (*) (参数列表)**

的指针变量。如，

```
void f(int x)
{
    ...
}
```



函数类型为： **void (int)**

➔ 函数指针的类型为： **void (*) (int)**;

➔ 声明函数指针变量： **void (*pf) (int)**;

- ◆ 函数类型仅由**函数头**规定。
- ◆ **函数名**是指向**函数实体**的指针，其**类型就是函数指针类型**，可为同类型的函数指针变量赋值，从而使函数指针变量指向函数。

最后注意:

- ◆ **函数名是地址常量**，不能被修改；
- ◆ **函数指针变量是变量**，可被赋予不同的值而指向不同的函数。
- ◆ 同类型的函数指针变量间可以相互赋值。

如:

```
int f1(int x)
{
    return x*x;
}
```

```
int f2(int x)
{
    return x+x;
}
```

```
int y, z;
void (*pf) (int );
pf = f1;
y=pf(5);
.....
pf=f2;
z=pf(5);
```

f1 = f2; ❌

f2 = pf; ❌

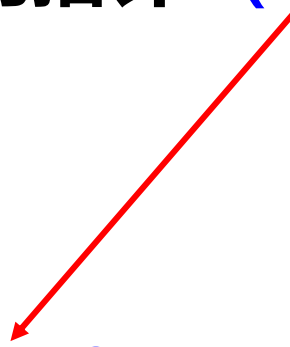
```
void (*pf2) (int );
pf2 = pf; ✓
```

注：给 pf 赋不同值(函数名)，pf 就指向不同的函数，调用时也就实现不同的功能。

8.9 指针和多维数组

8.9.1 一维数组和指向数组元素的指针 (一级指针)

设有**基本类型T** (如 int) ,

- ◆ 声明**基本元素**: **T x;**
- ◆ 声明**指向基本元素的指针**: **T * p; p=&x;**
- ◆ 声明**基本类型的一维数组**: **T a[3];**
 - 一维数组中的每个元素都是T类型的;
 - 每个元素的地址都是 **T *** 类型的;
 - **一维数组的数组名**就是首元素(a[0])的地址, **类型是: T ***
 - **p=a**后, p指向一维数组的**第一个元素**。

用“数组类型”的观点进一步看待一维数组的定义：

设有 T 类型一维数组的声明：**T a[3];**

根据 7.3.5 节（用“数组类型”的观点看待数组），首先可以将

T [3]

看作是一种**特定的“一维数组类型”**：**该类型是包含三个 T 类型元素的一维数组类型。**

其次，对于声明 **T a[3]** 则是定义了一个属于 **T [3]** 类型的变量 a，a 是这个“**集合变量**”的名字。

- ◆ **a 代表着由 3 个 T 型元素组成的一维数组的整体。**
- ◆ **a 是指向一维数组第一个元素的指针，类型是 T *。**

可以扩展到任意维的数组

如，设有 T 类型二维数组的声明：**T b[2][3];**

用“数组类型”的观点，可以将 **T [2][3]** 看作是一种**特定**的“**二维数组类型**”：**该类型是包含两行、三列共6个T类型基本元素的二维数组类型。**

而声明 **T b[2][3]** 则定义了一个属于 **T [2][3]** 类型的变量**b**，**b**是这个“**集合变量**”的名字。**b**代表着这个二维数组的整体。

问题：C语言规定，数组名是数组的起始地址，b是上述二维数组的起始地址，也是指针。那么**b指向谁呢？**

b的类型又是什么呢？

再如，设有 T 类型三维数组的声明：**T c[2][3][4];**

用“数组类型”的观点，可以将 **T [2][3][4]** 看作是一种**特定的“三维数组类型”**：**该类型是包含两层、三行、四列共24个T类型基本元素的三维数组类型。**

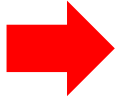
而声明 **T c[2][3][4]** 则定义了一个属于 **T [2][3][4]** 类型的变量**c**，**c**是这个“**集合变量**”的名字。**c**同样代表着这个**三维数组的整体**。

问题：C语言规定，数组名是数组的起始地址，c是上述三维数组的起始地址，也是指针。那么**c指向谁呢？**

c的类型又是什么呢？

8.9.2 指向数组的指针

数组类型指针：对上述“**数组类型**”，可以定义它们的指针类型：

- ◆ 一维数组类型： **T [3]** 一维数组指针类型： **T (*) [3]**
- ◆ 二维数组类型： **T [2][3]**  二维数组指针类型： **T (*) [2][3]**
- ◆ 三维数组类型： **T [2][3][4]** 三维数组指针类型： **T (*) [2][3][4]**
- ◆

指向数组的指针变量：可以定义属于上述“**指针类型**”指针变量。

这样的指针变量称为**指向数组的指针变量**。

- ◆ **指向数组的指针变量的声明**：

T (*p1)[3];

T (*p2)[2][3];

T (*p3)[2][3][4];

p1、p2、p3分别是指向**T [3]型一维数组**、**T [2][3]型二维数组**、**T [2][3][4]型三维数组**
这样数据对象的指针变量。

对指向数组的指针变量的说明:

- ◆ 声明指向数组的指针变量的形式:

T (*p1)[3];

数组基本元素
的类型名

数组维度的
说明

说明指针变量名:

T (*p2)[2][3];

T (*p3)[2][3][4];

(*** 标识符**) 写在“数组类型”中间:

- ◆ * 将标识符说明为指针变量名
- ◆ 其基类型是括号外定义的“**数组类型**”

指向数组的指针变量指向的是一个数组的整体，需用一个数组的“**整体地址**”为指向数组的指针变量赋值。

赋值后，**指向数组的指针变量将指向整个数组**，而不是数组中的基本元素 —— **如何赋值？**

8.9.2 二维数组和指向一维数组的指针

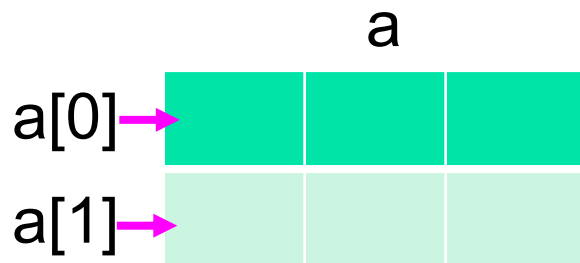
设声明 **T** 类型的二维数组：**T a[2][3];**

根据 **7.3.5节**的描述，a 可视为由两个一维数组组成的“**超一维数组**”，其元素是：“**类型为 T [3] 的一维数组**”。

- ◆ **a[0]**是“超一维数组”中第一个元素：

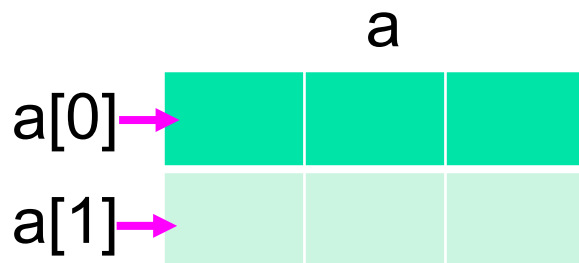
注：a[0]代表“超一维数组”中**第一个一维数组的整体**。

- ◆ 同时，**a[0]**可被视为这第一个一维数组的“**数组名**”。
- ◆ 因此，**a[0]**是指向本一维数组第一个元素的指针；
- ◆ **a[0]**值的类型是 **T ***；



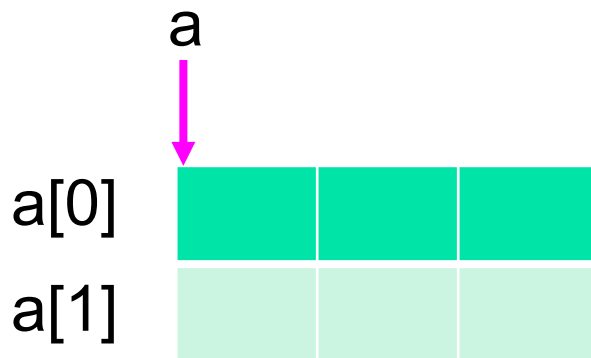
对a[1]，有同样的描述：

- ◆ a[1]是“超一维数组”中第二个元素：
- ◆ a[1]代表“超一维数组”中第二个一维数组的整体。
- ◆ a[1]被视为这第二个一维数组的“数组名”。
- ◆ a[1]值的类型是 **T ***；
- ◆ a[1]是指向本一维数组（“超一维数组”中第二个一维数组）第一个元素的指针；



另外，从**一维数组的角度**，**一维数组名**被视为指向**一维数组中第一个元素的指针**。那么，对于上述**“超一维数组” a**，

- ◆ 数组名 **a** 也是指向它的第一个元素 **a[0]** 的指针。而 **a[0]** 是一个一维数组，所以 **a 就是指向一维数组的指针**；
- ◆ **a[0]** 是类型为 **T * 指针**（实体是 **T [3]**），所以 **a 是一个指向指针的指针——二级指针（T**）**，具体类型是 **T (*) [3]**。



二维数组名作为二维数组的起始地址，其本质是一个指向一维数组的指针。其类型是 $T (*) [3]$ 。

因此,

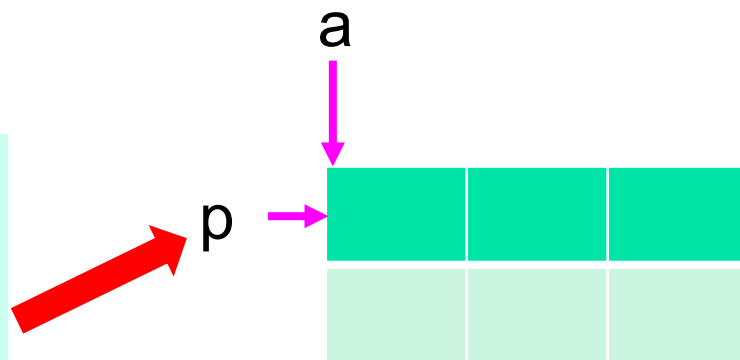
- ◆ 对于 T 类型的二维数组: **T a[2][3];**
 - 数组名 **a** 是指向一个有三个基本元素的一维数组的指针, 其值就是这个一维数组的“整体地址”。
- ◆ 设有指向“有三个基本元素的一维数组”的指针变量声明:

T (*p)[3];

那么, 可以用二维数组名 **a** 为 **p** 赋值:

p = a;

p 和 **a** 是相同类型的指针,
赋值后 **p** 指向 **a** 数组的第一行



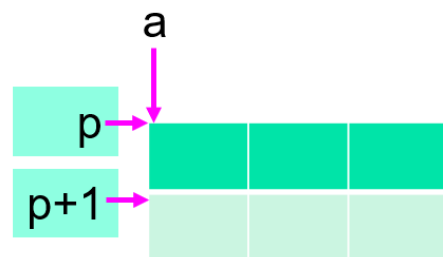
指向一维数组的指针的运算

设有：T 类型的二维数组：**T a[2][3];**

及相应的指向一维数组的指针变量：**T (*p)[3];**

令：**p=a;** /*p 指向 a 数组的第一行*/

问 p+1 等于多少?



- ◆ **p 加 1** 的地址增量由 **p 所指对象的数据长度** 决定。

对于类型 **T [3]**，数据长度为 **3*sizeof(T)**，所以

$$|p+1| = |p| + 3 * \text{sizeof}(T)$$

结果是：“**下一行的地址**”。

- ◆ 同理：**p-1** 将得到上一行的地址（如果这些行合法的话）

如, int型的二维数组: `int a[2][3];`

指向数组的指针变量: `int (*p)[3];`

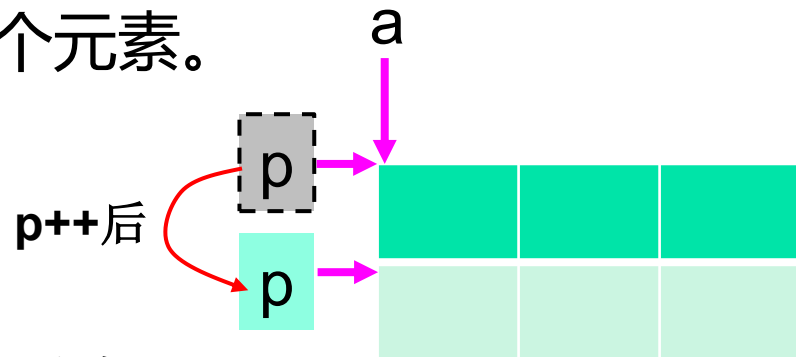
令: `p=a;` /*p指向a数组的第一行*/

$$\begin{aligned} \text{则 } |p+1| &= |p| + 3 * \text{sizeof(int)} \\ &= |p| + 12 \end{aligned}$$

32位系统

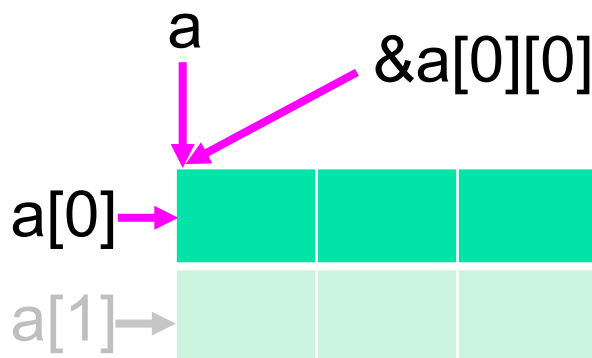
同理, 若开始有: `p=a;`

则 `p++`: 将使 p 指向 a 的第二行: 是指指向第二行整体,
而非第二行的第一个元素。



`p--`: 将使 p 指向前一行 (如果存在话)。

对二维数组的起始之处，可从三个角度描述该处的地址：



- ◆ 二维数组的起始地址：**a**，类型是 **T (*)[3]**
 - ◆ 二维数组中首行的起始地址：**a[0]**，类型是 **T***
 - ◆ 二维数组中第一个基本元素的地址：**&a[0][0]**，类型是 **T***
- 相同

以上三者的**值都是相同的**，指示的位置也相同，但它们的类型不一样，不能简单混为一谈。

基于指向一维数组的指针访问二维数组

- ◆ 访问指向基本元素的指针，得到的对象是基本元素：

如：T x, *q=&x; 则 ***q 等于 x**。

或：T b[5], *q=b; 则 ***q 等于 b[0]**。

若有：T a[2][3], T (*p)[3]=a;

则， ***p 等于a[0]**

a 是由 **a[0]**、**a[1]** 两个“超元素”组成的“超一维数组”；

p 是指向 **a** 的第一个“超元素”的指针。
所以 ***p** 得到是“超元素”本身，即第一行一维数组 **a[0]** 的实体。

****p 等于a[0][0]**

****p** 等价于 ***(p)**，进而等价于 ***(a[0])**，而 **a[0]** 是 **a** 的第一行一维数组的数组名，是指向第一行一维数组中第一个元素的指针，所以 ***(a[0])=a[0][0]**。

- ◆ 由于**二维数组名**本身就是指向其第一个“超元素”的指针，所以基于数组名亦有：

a** $\longleftrightarrow$ **a[0]** *a** $\longleftrightarrow$ **a[0][0]**

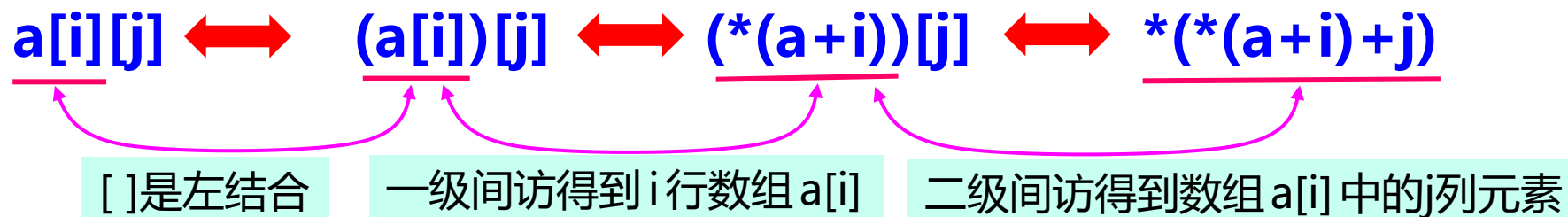
- ◆ 其它运算：（设开始时有 **p=a**）

p+1
a+1 $\left. \vphantom{\begin{matrix} p+1 \\ a+1 \end{matrix}} \right\}$ 计算得到 a 中第二行一维数组的指针（指数组的
整体地址）

***(p+1)**
***(a+1)** $\left. \vphantom{\begin{matrix} *(p+1) \\ *(a+1) \end{matrix}} \right\}$ 间访后得到 a 中第二行一维数组的**实体**，即 **a[1]**

**** (p+1)**
**** (a+1)** $\left. \vphantom{\begin{matrix} ** (p+1) \\ ** (a+1) \end{matrix}} \right\}$ 二级间访后得到 a 中第二行一维数组的第一个
元素，即 **a[1][0]**

对二维数组元素 $a[i][j]$ 的引用



若有 $p=a$ ，则基于指向 a 第一个“超元素”的指针 p 访问二维数组元素等价于基于数组名 a 访问二维数组元素，即：

$$p[i][j] \longleftrightarrow a[i][j]$$

$$p[i][j] \longleftrightarrow (p[i])[j] \longleftrightarrow (* (p+i))[j] \longleftrightarrow *(* (p+i) + j)$$

```
#include <stdio.h>
```

```
#define N 3
```

```
#define M 4
```

```
int main(void)
```

```
{
```

```
    int i, j, a[N][M];
```

```
    int (*p) [3]=a;
```

```
    printf("输入数组元素: \n");
```

```
    for(i=0; i<N; i++)
```

```
        for(j=0; j<M; j++)
```

```
            scanf("%d", &p[i][j]);    /*基于指向 a 中第一行一维数组的指针访问*/
```

```
    printf("输出数组元素: \n");
```

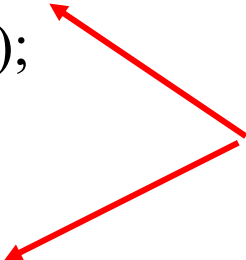
```
    for(i=0; i<N; i++)
```

```
        for(j=0; j<M; j++)
```

```
            scanf("%d", a[i][j]);    /*用数组名访问*/
```

```
    return 0;
```

```
}
```



这里 **p[i][j]** 与 **a[i][j]** 完全等价

若有: $T\ a[2][3],\ T\ (*p)[3]=a;$

(1) 以下表达式等价, 都是**指向二维数组a中首行一维数组的指针**:

$a\quad p$

(2) 以下表达式等价, 都是取得二维数组a中**首行一维数组的实体**:

$a[0]\quad *a\quad *(a+0)\quad *p\quad p[0]\quad *(p+0)$

(3) 以下表达式等价, 都是**取得二维数组a第二行一维数组的指针**:

$a+1\quad p+1$

(4) 以下表达式等价, 都是**取得二维数组a第二行一维数组的实体**:

$a[1]\quad *(a+1)\quad *(p+1)\quad p[1]$

若有: $T\ a[2][3], T\ (*p)[3]=a;$

(5) 以下表达式等价, 都是引用二维数组a中首行一维数组的第一个元素 $a[0][0]$:

$a[0][0]\quad *(a[0]+0)\quad *a[0]\quad **a\quad p[0][0]\quad *p[0]\quad **p$

(6) 以下表达式等价, 都是取得二维数组a第二行一维数组第一个元素 $a[1][0]$:

$a[1][0]\quad *(a[1]+0)\quad *(p[1]+0)\quad p[1][0]\quad **(a+1)\quad **(p+1)$

(7) 以下表达式等价, 都是取得二维数组a中i行j列的元素 $a[i][j]$:

$a[i][j]\quad *(a[i]+j)\quad *(* (a+i)+j)$

$p[i][j]\quad *(p[i]+j)\quad *(* (p+i)+j)$


```
#include <stdio.h>
```

```
#define N 3
```

```
#define M 4
```

```
int main(void)
```

```
{
```

```
    int i, j, a[N][M];
```

```
    int (*p) [3]=a;
```

```
    printf("输入数组元素: \n");
```

```
    for(i=0; i<N; i++)
```

```
        for(j=0; j<M; j++)
```

```
            scanf("%d", &p[i][j]);
```

等价

```
scanf("%d", p[i]+j);
```

```
    printf("输出数组元素: \n");
```

```
    for(i=0; i<N; i++)
```

```
        for(j=0; j<M; j++)
```

```
            scanf("%d", a[i][j]); return 0;
```

等价

```
scanf("%d", *(a[i]+j));
```

```
}
```

```
scanf("%d", *(*(a+i)+j));
```

8.9.3 N维数组和指向低一级数组的指针

设声明 **T** 类型的 **N** 维数组 **x**: **T x[N1][N2]...[NN];**

T 是 **N** 维数组 **x** 中基本元素的类型

根据 **7.3.5节** 的描述, **x** 可视为由 **N1** 个一维数组组成的“**超一维数组**”, 其元素是: “类型为 **T [N2]...[NN]** 的 **N-1 维数组**”

—— 这里, 把**比N维少一维**、且各维大小等于原**N维数组**从第二维开始各维大小的数组称为这个**N维数组**的**低一级数组**。

则, **x** 就是由**比其低一级的 N-1 维数组**组成的“**超一维数组**”;

- ◆ **x[0]** 是该“超一维数组”的第一个元素, 是一个 **N-1 维数组**;
- ◆ **x[1]** 是该“超一维数组”的第二个元素, 也是一个 **N-1 维数组**;
- ◆

将 $T[N2]...[NN]$ 视为一种 “**N-1维数组类型**”，则可以定义该类型的指针类型（即指向N-1维数组的指针类型）：

$T(*)[N2]...[NN]$

- ◆ X是由N1个 “ $T[N2]...[NN]$ 型数组” 组成的“**超一维数组**”，所以 **x 是指向 $T[N2]...[NN]$ 型数组的指针**，类型就为：

$T(*)[N2]...[NN]$

- ◆ 定义 $T(*)[N2]...[NN]$ 型的指针变量p：

$T(*p)[N2]...[NN];$

则可有： $p=x;$ （即令 p 指向 x 的第一个“超元素”：x 的第一个 $T[N2]...[NN]$ 型 N-1 维数组）

—— p 称为**指向（比x）低一级数组的指针**。

如，设有三维数组 $T\ x[2][3][4]$

$T\ [3][4]$ 是一个二维数组类型，定义该类型的指针类型：

$T\ (*)\ [3][4]$

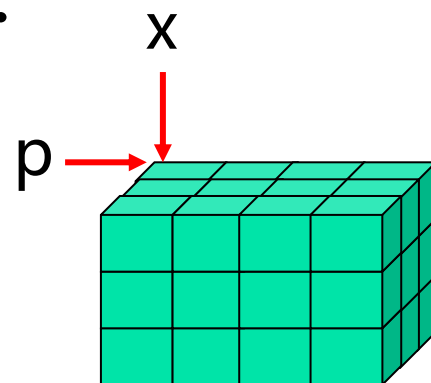
- ◆ x 是由2个 " $T\ [3][4]$ 型二维数组" 组成的"超一维数组"；
- ◆ x 是指向 " $T\ [3][4]$ 型"二维数组的指针，类型为 $T\ (*)\ [3][4]$ ；
- ◆ 定义指向 $T\ [3][4]$ 型二维数组的指针变量 p ：

$T\ (*)\ p\ [3][4];$

并令： $p=x;$

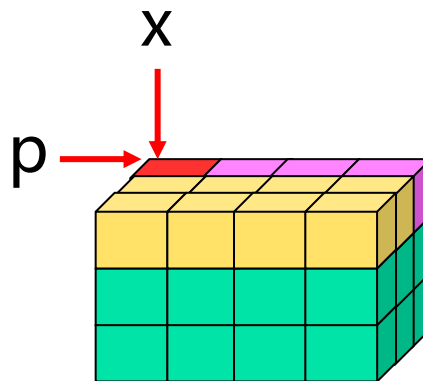
则： p 是指向 x 的第一个“超元素”的指针：

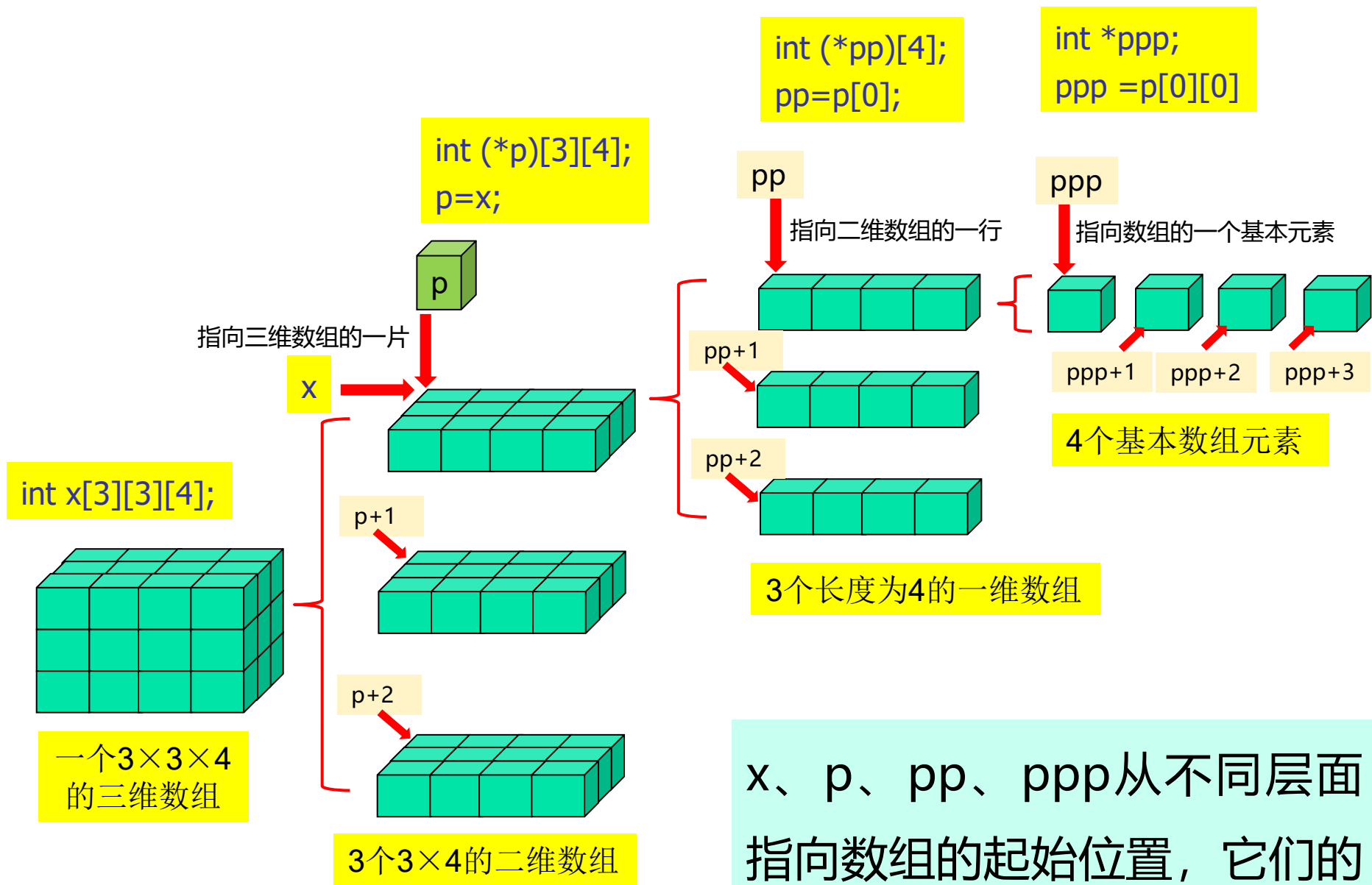
p 指向 x 的第一个 $T\ [3][4]$ 型二维数组。



基于: $p=x$; 进一步有:

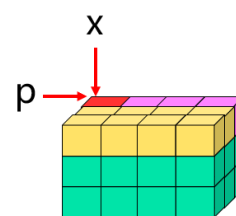
- ◆ $*p$: 得到 x 的第一层的 3×4 二维数组 $x[0]$ (实体);
同时也是指向第一层第一行的一维数组的指针,
类型是 $T*[3]$;
- ◆ $**p$: 得到 x 的第一层第一行的一维数组 $x[0][0]$ (实体),
同时也是指向第一层第一行的一维数组中第一个基本元素的指针, 类型是 $T*$;
- ◆ $***p$: 得到 x 的第一层第一行的一维数组中第一个基本元素 $x[0][0][0]$, 类型是 T 。





x、p、pp、ppp从不同层面指向数组的起始位置，它们的值相同，但“**类型**”不同。

若有: $\text{T } x[2][3][4], \text{T } (*p)[3][4]=x;$



(1) 以下表达式等价, 都是**指向三维数组a中首层二维数组的指针**:

x p

(2) 以下表达式等价, 都是**取得三维数组a中首层二维数组的实体**:

$x[0]$ $*x$ $*(x+0)$ $*p$ $p[0]$ $*(p+0)$

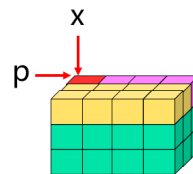
(3) 以下表达式等价, 都是**取得三维数组a第二层二维数组的指针**:

$x+1$ $p+1$

(4) 以下表达式等价, 都是**取得三维数组a第二层二维数组的实体**:

$x[1]$ $*(x+1)$ $*(p+1)$ $p[1]$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$



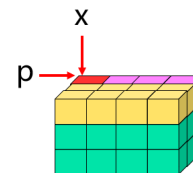
(1) 以下表达式等价, 都是**指向三维数组a中首层二维数组的第一行的指针**: $x[0]$ $p[0]$

(2) 以下表达式等价, 都是**取得三维数组a中首层二维数组的第一行的实体**: $x[0][0]$ $*(x[0]+0)$ $*(x)$ $**x$
 $p[0][0]$ $*(p[0])$ $*(p)$ $**p$

(3) 以下表达式等价, 都是**取得三维数组a中首层二维数组的第二行的指针**: $x[0]+1$ $*x+1$ $p[0]+1$ $*p+1$

(4) 以下表达式等价, 都是**取得三维数组a中首层二维数组的第二行的实体**: $x[0][1]$ $*(x[0]+1)$ $*(x+1)$
 $p[0][1]$ $*(p[0]+1)$ $*(p+1)$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$



(1) 以下表达式等价, 都是**指向三维数组a中第二层二维数组的**

第一行的指针: $x[1]\quad *(x+1)\quad p[1]\quad *(p+1)$

(2) 以下表达式等价, 都是**取得三维数组a中第二层二维数组的**

第一行的实体: $x[1][0]\quad *(x[1])\quad *(* (x+1)+0)\quad *(* (x+1))$

$p[1][0]\quad *(p[1])\quad *(p[1]+0)\quad *(* (p+1))$

(3) 以下表达式等价, 都是**取得三维数组a中第二层二维数组的**

第二行的指针: $x[1]+1\quad *(x+1)+1\quad p[1]+1\quad *(p+1)+1$

(4) 以下表达式等价, 都是**取得三维数组a中第二层二维数组的**

第二行的实体: $x[1][1]\quad *(x[1]+1)\quad *(* (x+1)+1)$

$p[1][1]\quad *(p[1]+1)\quad *(* (p+1)+1)$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(1) 以下表达式等价, 都是指向三维数组a中第一层二维数组的第一行首元素的指针 $\&x[0][0][0]$:

$\&x[0][0][0]$

$x[0][0]\quad *(x[0]+0)\quad *(*x+0)\quad **x$

$p[0][0]\quad *(p[0]+0)\quad *(*p+0)\quad **p$

(2) 以下表达式等价, 都是取得三维数组a中第一层二维数组的第一行首元素的实体 $x[0][0][0]$:

$x[0][0][0]\quad *(x[0][0])\quad *(* (x[0]+0))\quad *(* (*x))\quad ***x$

$p[0][0][0]\quad *(*p[0][0])\quad *(* (p[0]+0))\quad *(* (*p))\quad ***p$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(3) 以下表达式等价, 都是指向三维数组a中第一层二维数组的第一行第二元素的指针 $\&x[0][0][1]$:

$x[0][0]+1$ $*(x[0]) + 1$ $*(x)+1$ $**x+1$

$p[0][0]+1$ $*(p[0]) + 1$ $(*p)+1$ $**p+1$

(4) 以下表达式等价, 都是取得三维数组a中第一层二维数组的第一行第二个元素的实体 $x[0][0][1]$:

$x[0][0][1]$ $*(x[0][0]+1)$ $*(x[0]+1)$ $*(x)+1$ $**x+1$

$p[0][0][1]$ $*(p[0][0]+1)$ $*(p[0]+1)$ $(*p)+1$ $**p+1$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(5) 以下表达式等价, 都是**指向三维数组a中第一层二维数组的第二行首元素的指针** $\&x[0][1][0]$:

$\&x[0][1][0]$

$x[0][1]\quad *(x[0]+1)\quad *(*x+1)$

$p[0][1]\quad *(p[0]+1)\quad *(*p+1)$

(6) 以下表达式等价, 都是**取得三维数组a中第一层二维数组的第二行首元素的实体** $x[0][1][0]$:

$x[0][1][0]\quad *(x[0][1])\quad *(* (x[0]+1))\quad *(* (*x+1))$

$p[0][1][0]\quad *(*p[0][1])\quad *(* (p[0]+1))\quad *(* (*p+1))$

若有: $\text{T } x[2][3][4], \text{T } (*p)[3][4]=x;$

(7) 以下表达式等价, 都是指向三维数组a中第一层二维数组的第二行第二元素的指针 $\&x[0][1][1]$:

$x[0][1]+1 \quad *(x[0]+1)+1 \quad *(*x+1)+1$

$p[0][1]+1 \quad *(p[0]+1)+1 \quad *(*p+1)+1$

(8) 以下表达式等价, 都是取得三维数组a中第一层二维数组的第二行第二个元素的实体 $x[0][1][1]$:

$x[0][1][1] \quad *(x[0][1]+1) \quad *(*x[0]+1)+1 \quad *(*(*x+1)+1)$

$p[0][1][1] \quad *(p[0][1]+1) \quad *(*p[0]+1)+1 \quad *(*(*p+1)+1)$

若有： $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(1) 以下表达式等价，都是指向三维数组a中第二层二维数组的第一行首元素的指针 $\&x[1][0][0]$ ：

$\&x[1][0][0]$

$x[1][0]\quad *(x[1]+0)\quad *(* (x+1))$

$p[1][0]\quad *(p[1]+0)\quad *(* (p+1))$

(2) 以下表达式等价，都是取得三维数组a中第二层二维数组的第一行首元素的实体 $x[1][0][0]$ ：

$x[1][0][0]\quad *(x[1][0])\quad *(* (x[1]+0))\quad *(* (* (x+1)))$

$p[1][0][0]\quad *(* p[1][0])\quad *(* (p[1]+0))\quad *(* (* (p+1)))$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(3) 以下表达式等价, 都是指向三维数组a中第二层二维数组的第一行第二个元素的指针 $\&x[1][0][1]$:

$\&x[1][0][1]$

$x[1][0]+1$ $*(x[1]+0)+1$ $*(*(x+1))+1$ $** (x+1)+1$

$p[1][0]+1$ $*(p[1]+0)+1$ $*(*(p+1))+1$ $** (p+1)+1$

(4) 以下表达式等价, 都是取得三维数组a中第二层二维数组的第一行第二个元素的实体 $x[1][0][1]$:

$x[1][0][1]$ $*(x[1][0]+1)$ $*(*(x[1]+0)+1)$ $*(** (x+1))+1$

$p[1][0][1]$ $*(p[1][0]+1)$ $*(*(p[1]+0)+1)$ $*(** (p+1))+1$

若有: $T \ x[2][3][4], T \ (*p)[3][4]=x;$

(5) 以下表达式等价, 都是指向三维数组a中第二层二维数组的第二行首元素的指针 $\&x[1][1][0]$:

$x[1][1]+0 \quad *(x[1]+1) \quad *(*x+1)+1$

$p[1][1]+0 \quad *(p[1]+1) \quad *(*p+1)+1$

(6) 以下表达式等价, 都是取得三维数组a中第二层二维数组的第二行首元素的实体 $x[1][1][0]$:

$x[1][1][0] \quad *(x[1][1]+0) \quad *(*x[1]+1) \quad *(*(*x+1)+1)$

$p[1][1][0] \quad *(p[1][1]+0) \quad *(*p[1]+1) \quad *(*(*p+1)+1)$

若有: $T\ x[2][3][4],\ T\ (*p)[3][4]=x;$

(7) 以下表达式等价, 都是**指向三维数组a中第二层二维数组的第二行第二个元素的指针** $\&x[1][1][1]$:

$x[1][1]+1$ $*(x[1]+1)+1$ $((*(x+1)+1)+1)$

$p[1][1]+1$ $*(p[1]+1)+1$ $((*(p+1)+1)+1)$

(6) 以下表达式等价, 都是**取得三维数组a中第二层二维数组的第二行第二个元素的实体** $x[1][1][1]$:

$x[1][1][1]$ $*(x[1][1]+1)$ $((*(x[1]+1)+1))$ $((*(* (x+1)+1)+1)$

$p[1][1][1]$ $*(p[1][1]+1)$ $((*(p[1]+1)+1))$ $((*(* (p+1)+1)+1)$

8.9.3 二维数组参数的指针表示

(1) 函数的参数可以是二维数组形式

例8.22 定义函数aveCols，计算并输出二维数组**各列元素**的平均值。

```
void aveCols1(int a[ ][5], int row)
```

```
{
```

以二维数组作为函数的参数

```
    int c, r, sum;
```

```
    for(c=0; c<5; c++) { /* 扫描各列 */
```

```
        for(sum=0; r=0; r<row; r++) /* 扫描各行 */
```

```
            sum += a[r][c]; /* 同一列的元素求和 */
```

```
        printf("col %d: average = %.1f\n",c, 1.0*sum/row);
```

```
}
```

(2) 用指向一维数组的指针作为函数参数

二维数组名是指向（第一行）一维数组的指针，因此二维数组作函数形式参数时，可以用指向一维数组的指针形式替换。

```
void aveCols2(int (*pa) [5], int row)
```

```
{
```

以指向一维数组的指针
作为函数的参数

```
int c, r, sum;
```

```
for(c=0; c<5; c++) {
```

```
    for(sum=0; r=0; r<row; r++)
```

```
        sum += *(*pa+r)+c);    /*二级间访，等价于 pa[r][c]*/
```

```
    printf("col %d: average = %.1f\n",c, 1.0*sum/row);
```

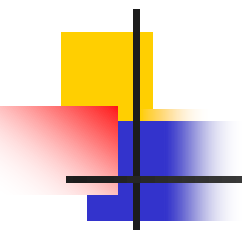
```
}
```

函数体内对指向一维数组的指针的使用和用二维数组名是完全一样的

- ◆ 调用时使用**二维数组名作实参**或使用**指向该二维数组第一行一维数组的指针作实参**完全等价。

```
void aveCols1(int a[ ][5], int row);  
void aveCols2(int (*pa) [5], int row);  
int main()  
{  
    int i, j, ary[4][5], (*p) [5] = ary;  
    for(i=0; i<4; i++)  
        for(j=0; j<5; j++)  
            scanf("%d", &ary[i][j]);  
    aveCols1(ary, 4);  
    aveCols2(p, 4);  
    return 0;  
}
```

址传递：把实参数组名ary的值或指针p的值（二者都是二维数组ary第一行一维数组的地址）复制给形参二维数组名a或形参数组指针pa。



设有二维数组 $T\ a[2][5]$ 。

虽然二维数组的数组名 a 是一个 “**二级指针**”，但不能简单理解为 a 是 T^{**} 的指针，而是一个 **$T(*)[5]$ 的指针**。

```
void aveCols2(int (*pa) [5], int row)
```

```
{
    int c, r, sum;
    for(c=0; c<5; c++) {
        for(sum=0; r=0; r<row; r++)
            sum += (*(pa+r)+c);    /*二级间访, 等价于pa[r][c]*/
        printf("col %d: average = %.1f\n",c, 1.0*sum/row);
    }
}
```

```
int main()
```

```
{
    int i, j, ary[4][5], (*p) [5] = ary;
    for(i=0; i<4; i++)
        for(j=0; j<5; j++)
            scanf("%d", &ary[i][j]);
    aveCols2(p, 4); ✓
    return 0;
}
```

```
void aveCols3(int **pa, int row)
```

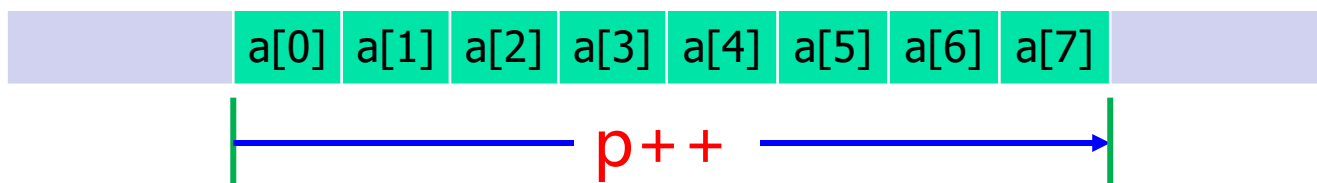
```
{
    int c, r, sum;
    for(c=0; c<5; c++) {
        for(sum=0; r=0; r<row; r++)
            sum += (*(pa+r)+c);
        printf("col %d: average = %.1f\n",c,
            1.0*sum/row);
    }
}
```

```
aveCols3(p, 4);
```



8.9.4 用指向基本元素的一级指针访问二维/多维数组的元素

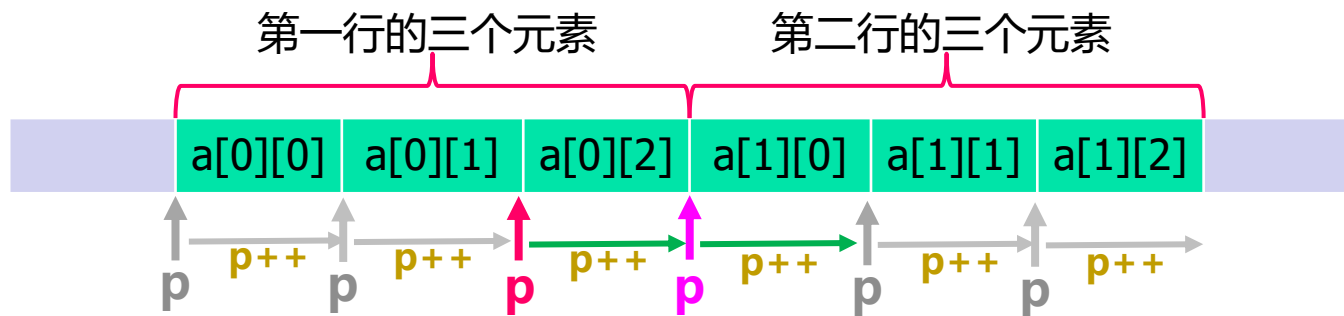
一维数组中的元素在内存中是“直接”线性排列的。如 `int a[8];`



定义一个指向基本元素的一级指针并令其指向一维数组的第一个元素，如 `int *p=a;` 然后可以利用 `p++` 的方式顺序访问到一维数组中的每一个元素。

```
int i, sum=0, a[8], *p=&a[0]; /* p指向a[0] */  
for(i=0; i<8; i++, p++)  
    sum = sum + *p;
```

二维数组中的元素虽然在**逻辑**上具有行、列结构，但按照“**行序为主序**”的存储方式，**所有元素在内存中也是“线性排列”**的：第一行元素后紧接着第二行元素、然后是第三行元素、……。如 **int a[2][3];**



物理存储上，第一行最后一个元素的后面是第二行的第一个元素，所有可以继续向后扫描

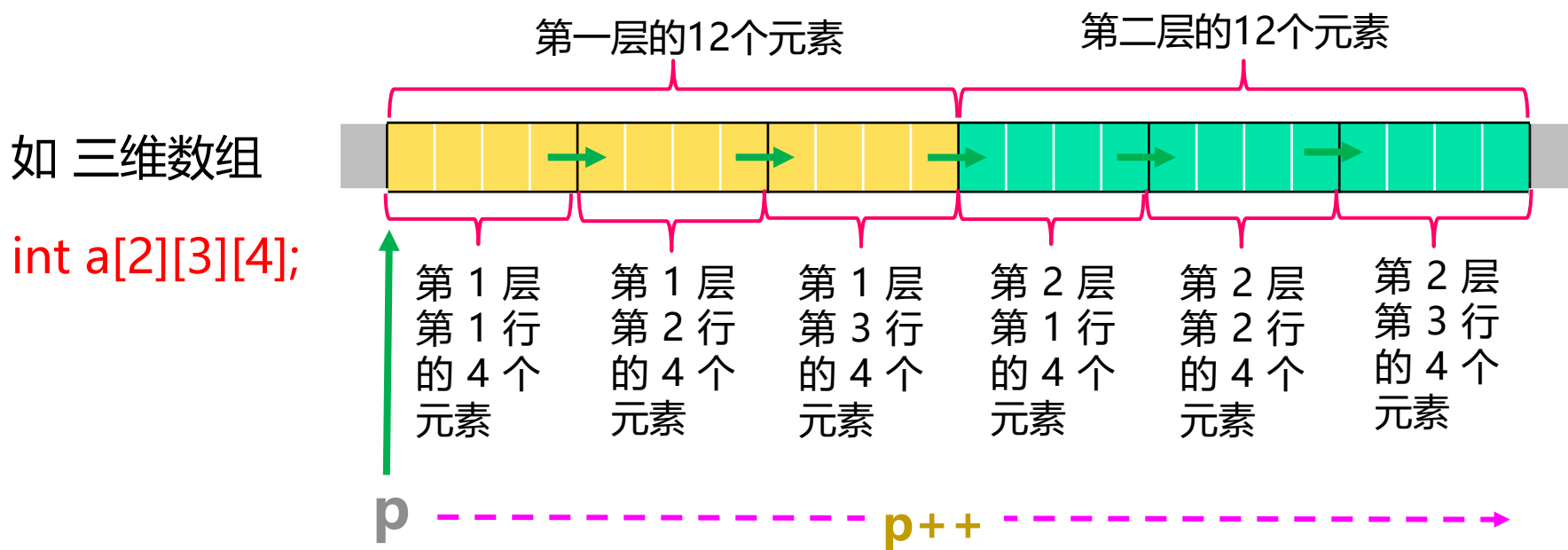
- ◆ 定义一个指向基本元素的一级指针，并令其指向整个二维数组的“**第一个元素**”，如 **int *p=&a[0][0];**
- ◆ 则利用**p++**的方式依然可以**顺序访问**到二维数组中的每一个元素。

```
int i, sum=0, a[2][3], *p=&a[0][0]; /* p指向a[0][0] */
```

```
for(i=0; i<2*3; i++, p++) sum = sum + *p; /*元素总数是2*3=6个*/
```


该性质对更高维的数组依然成立：高维数组的元素也都以“**行序为主序**”的方式在内存中“**线性排列**”。

元素在物理存储空间中是一个接着一个顺序排列的



定义一个指向基本元素的一级指针，并令其指向整个数组的第一个元素，如 `int *p=&a[0][0][0];` 则利用 `p++` 的方式就可以顺序访问到多维数组中的每一个元素。

也就是，逻辑上的多维结构在物理存储上元素却是线性排列的，物理上元素之间只有线性结构，所以可以利用：

指向基本元素的指针+1

的方式**线性扫描整个数组的所有元素**，而不用管原本数组的逻辑结构是多少维的。

如：

```
int i, sum=0, a[2][3][4], *p=a[0][0][0]; /* p指向a[0][0][0] */  
for(i=0; i<2*3*4; i++, p++) /*元素总数是2*3*4=24个*/  
    sum = sum + *p;
```

例 用**指向基本元素的指针**作为函数参数计算并输出二维数组**各列**
元素的平均值。

```
void aveCols3(int * a, int row, int col)
```

a是指向基本
元素的指针

```
{
```

```
    int c, r, sum;
```

```
    for(c=0; c<col; c++) {
```

```
        for(sum=0; r=0; r<row; r++)
```

```
            sum += *(a+r*col+c);
```

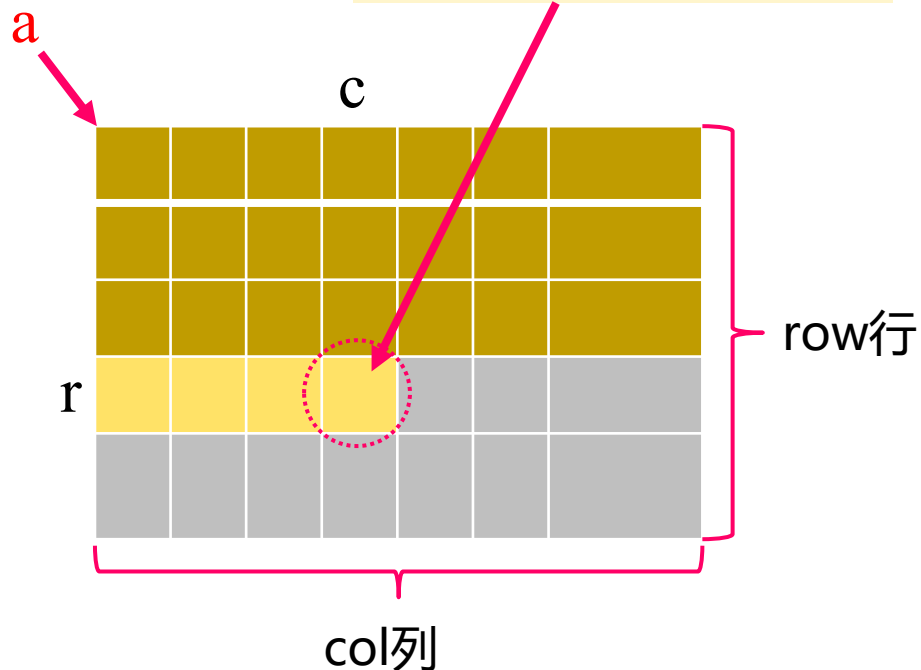
按照存储顺序计算
元素的地址

```
        printf("col %d: average = %.1f\n",c, 1.0*sum/row);
```

```
}
```

元素 $a[r][c]$ 相对于数组
开始位置的位置偏移是

$r*col+c$



8.10 动态存储分配

静态数据结构：是指程序中通过**变量或数组声明**的方式建立的数据结构，比如简单变量、数组等。它们在声明的同时分配存储，且一经分配，在执行过程中就不能再改变。

到目前为止所介绍的数据结构都是静态数据结构。

动态数据结构：是指在程序运行过程中通过调用系统提供的**动态存储分配函数**，**向系统申请而逐步建立起来的数据结构**。

- ◆ 动态数据结构可以根据应用的需要而建立，所占存储的大小可变，但需要通过向系统申请而获得，使用完后要释放，将所占存储“还给”系统。

C语言的动态存储分配函数

- ◆ **动态存储分配函数**是C语言的标准函数，函数的原型声明在头文件<stdlib.h>。使用动态存储分配函数要在程序里：

#include <stdlib.h>

- ◆ 主要学习下列与动态存储分配相关的函数。

void * malloc(size_t size);

void * calloc(size_t n, size_t size);

void * realloc(void * p_block, size_t size);

void free(void * p_block);

这里，**size_t**表示unsigned int，即无符号整型。它是C语言预定义的一个类型名：**typedef unsigned size_t;**

1、malloc 函数

函数原型： `void * malloc (size_t size);`

功 能：向系统**申请分配 size 个字节大小的连续存储区域**。

- ◆ 如果分配成功，返回所分配存储区域的**首地址**，类型是 `void *`；如果分配失败，**返回 NULL**。

注：

- (1) 所分配存储区域是内存中的一段**连续字节空间**，字节的个数等于 `size`，内容未被初始化，是**随机值**。
- (2) **其中第一个字节的地址是这段字节空间的首地址**，`malloc` **返回该地址值，但类型是 `void *`**，不能直接使用，**必须强制类型转化为具体指针类型才能使用该空间**。

(3) 动态申请的数据空间没有名字，**只有一个起始地址标识它的存在**。所以一般要**将该地址赋给一个相应的指针变量**，然后通过指针变量引用该空间里的数据对象。

例如，利用 malloc 函数动态创建一个有**6个元素的“整型数组”**。

分析：该整型数组不是通过数组声明（如int x[6]）的方式建立，而是要通过向系统**动态申请 6 个整型元素的连续字节空间**。

(1) 6 个整型元素的字节数量是 **6*sizeof(int)**，如32位系统中将是 $6*4=24$ 个。

(2) 用 malloc 申请 **6*sizeof(int)** 个字节的空间，并**将地址强制类型转化为 int ***，**赋给一个整型指针 p**。

具体语句如下：

```
int *p;
```

```
p=(int *)malloc(6*sizeof(int));
```

注：动态申请并不一定能成功，如系统内存已被用完或没有足够大的连续空间，则申请失败。此时 **malloc 返回 NULL**。所以**实际使用时，一般要在申请空间后，对指针是否为空进行判断**：

只能对非空指针进行后续处理，而指针为空时要做异常处理，否则会造成系统异常。如：

```
int * p = (int *)malloc(6*sizeof(int));
```

```
if(p)
```

```
    for(i=0; i<6; i++) p[i]=i;
```

```
else {
```

```
    printf("dynamic alloc failed!"); exit(-1);
```

```
}
```

说明：若p为空，说明系统已没有足够内存分配给程序，所以程序也没必要再继续执行，此时调用**exit(-1)**，强迫程序结束并返回操作系统。

exit()函数：C语言的一个库函数，功能是**结束当前程序的执行，立刻返回操作系统。程序中只要调用exit，就结束。**

exit和return的区别：

return 是结束一个函数的执行，除非该函数是 main 函数，才会从 main 函数“返回”操作系统，其它函数返回时，仅是返回到上一级主调函数中调用它的位置，然后继续执行其它语句。

而 **exit**，不论在哪里执行，一旦执行，将立刻结束整个程序的执行，然后返回操作系统。多数情况下代表着“**程序执行失败**”（正常情况下程序都是从 main 函数里结束，而 exit 通常在处理异常时使用）。

exit的参数：int型

是给操作系统的一个返回值，表示程序的执行状态。通常 **exit(0)** 表示程序正常退出，**非零时代表异常**，操作系统可以根据这个状态值对程序的执行情况进行管理。

2、calloc 函数

函数原型： `void * calloc(size_t n, size_t size);`

功能： 向系统**申请分配 n 个单元、每单元的大小为 size 个字节。**

- ◆ 如果分配成功，返回所分配存储区域的**首地址**；
- ◆ 如果分配失败，则返回 **NULL**。

calloc的处理是： (1) 按照 **n*size** 计算总字节数量，然后在内存中分配这个数量的**连续字节空间**，返回其首址，但字节空间内依然是**流式空间**，没有内部逻辑结构，“**不逻辑分段**”； (2) 与 malloc 函数不同的是，**calloc 对其申请的空间做初始化；全部置 0**；

如，利用 calloc 函数动态创建一个有 6 个元素的整型数组。

```
p = (int *) calloc(6, sizeof(int));
```

分配到的存储区域被自动初始化为 0。

3、realloc 函数

函数原型： `void * realloc(void *p_block, size_t size);`

功 能： **对指针 p_block 所指向的已是动态分配的存储区域重新进行动态存储分配。** p_block 指向曾由 malloc 函数或 calloc 函数分配的存储区域； size 是申请重新分配存储的字节数。

- ◆ 如果分配成功，返回新分配存储区域的首地址；
- ◆ 如果分配失败，则返回NULL。

- 如果新分配的存储区比原分配的存储区小，则**新分配的存储区中仍然是原来老存储区中“这部分”的数据（复制过来的）**；
- 如果新分配的存储区比原分配的存储区大，则新分配的存储区中**新增部分的区域未被初始化。**

例，基于上面申请的 6 个元素的整型数组（p[0] ~ p[5]分别赋值 0~5），增加下面语句，重新进行动态存储分配：

(1) `realloc((int *)p, 3*sizeof(int));`

- ◆ 返回时，p 指向重新分配的 3 个整型数据单元；
- ◆ 新分配的存储区比原分配的存储区小；
- ◆ p[0]、p[1]、p[2] 的值等于原来的 0、1、2。

(2) `realloc((int *)p, 8*sizeof(int));`

- ◆ 返回时，p 指向重新分配的 8 个整型数据单元；
- ◆ 新分配的存储区比已分配的存储区大；
- ◆ p[0] ~ p[5] 等于原来的 0 ~ 5，但 p[6] 和 p[7] 为随机值。

4、free 函数

函数原型: **void free(void * p_block);**

功 能: 释放由指针 **p_block** 所指向的存储区。

- ◆ **p_block** 所指的应是由 **malloc**, **calloc** 或 **realloc** 分配的存储区;
- ◆ **一旦释放, 就不能再用 p_block 引用存储区中的数据**, 否则会出现意想不到的错误。
- ◆ **free 函数无返回值**, 返回值类型为 **void**。
- ◆ **如果 p_block 为 NULL, free 函数无释放操作。**

由 **malloc**, **calloc** 或 **realloc** 分配的存储区, **使用完后一定要通过 free 函数释放**, 将存储区交还给操作系统, 否则就会造成内存垃圾 (**内存泄漏, memory leak**) 。

例，函数 GetAry(n) 申请参数 n 指定大小的**动态整型数组**空间，
返回首地址，然后在 main 函数中使用该动态数组空间。

```
int main()
```

```
{
```

```
    int n, i, *p;
```

```
    printf("Input n:");
```

```
    scanf("%d",&n);
```

```
    p = GetAry(n);
```

```
    if(p) {
```

```
        printf("Input n numbers:");
```

```
        for(i=0;i<n;i++) scanf("%d", &p[i]); /*为 “数组” 输入元素*/
```

```
        for(i=0;i<n;i++) printf("%d", p[i]); /*输出 p 中的各元素*/
```

```
        free(p);
```

```
    }
```

```
}
```

```
int * GetAry(int n)
```

```
{
```

```
    if(n==0) return NULL;
```

```
    return (int *) malloc(n*sizeof(int));
```

```
}
```

free 时直接用指针变量即可

说明:

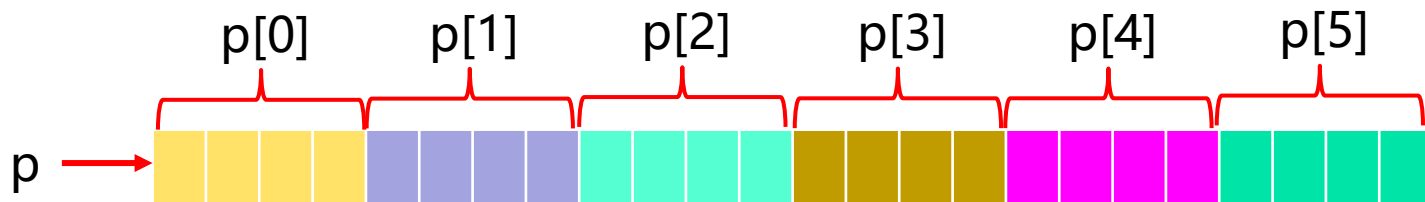
(1) malloc 函数成功执行后, 将返回的内存空间首地址**类型强制后赋给特定类型的指针变量**, 如:

```
int *p = (int *) malloc(6*sizeof(int));
```

之后基于**指针变量的基类型来解释数据空间中的对象并访问**。

如这里, p 是整型指针, 则基于 p 访问该空间就是对一个**整型数组**的访问:

空间在逻辑上将被视为分成 6 段, 每段 4 字节, 对应一个整型数据单元。根据指针的使用规则, p[0]~p[5] 就是其中的 6 个元素, 从而**可以象使用数组一样使用这段由 malloc 分配到的空间**。



(2) **程序员必须控制访问动态空间的有效范围**，不要超范围使用，否则会造成严重后果。

```
如： int * p  
      p=(int *) malloc(6*sizeof(int));  
      for(i=0;i<1000;i++)  
          p[i]=i;
```

这里尽管**看起来有空间的大小的指示**，如 `6*sizeof(int)`，但编译器只是使用这个数字分配相应大小的空间，并没有将空间大小的信息反馈给 `p` 来控制这块空间的安全使用，也不检查空间的越界访问。**对 `p` 而言，也只是代表 `malloc` 申请到的空间的起始地址，不知道空间的大小。它只管从这个地址使用，越不越界不检查。**总之，**空间的安全使用要由程序员自己负责。**

(3) 申请的空间，使用完一定要及时释放，**防止产生内存垃圾**。

如下所示：

如： `int * p`

`p=(int *) malloc(6*sizeof(int));`

`p=(int *) malloc(100*sizeof(int));`

由于**动态申请的数据空间没有名字**，只有地址标识。如上，若尚未释放 p 先前所指的 6 个整数数据空间，就又令其指向新的地址，则它先前所指的空间就“**丢了**”，而且再也找不回了，成为**内存垃圾**——不仅自己再也不能用，操作系统也无法回收，造成内存浪费。因此**在改变一个指向动态内存空间的指针指向之前，要确保该段内存被释放或者被其它指针变量所指，不要“丢失”**。

应用示例：由字符指针数组动态创建“字符串数组”

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define MAXLEN 80
```

```
#define STRNUM 10
```

```
int main()
```

```
{
```

```
    int i;
```

```
    char * s[STRNUM], temp[MAXLEN];
```

```
    printf("输入%d个字符串，一行1个字符串", STRNUM);
```

```
    for(i=0; i<STRNUM; i++) {
```

```
        gets(temp);    /*在不知道输入长度的情况下，先用temp接收输入的字符串*/
```

```
        s[i]=(char *)malloc(strlen(temp)+1); /*然后根据输入的字符串长度申请一个大小最合适的空间*/
```

```
        if(s[i]==NULL) return -1;    /*如果空间申请异常，立刻终止程序的执行*/
```

```
        strcpy(s[i], temp);    /*将temp中的字符串拷入新空间中*/
```

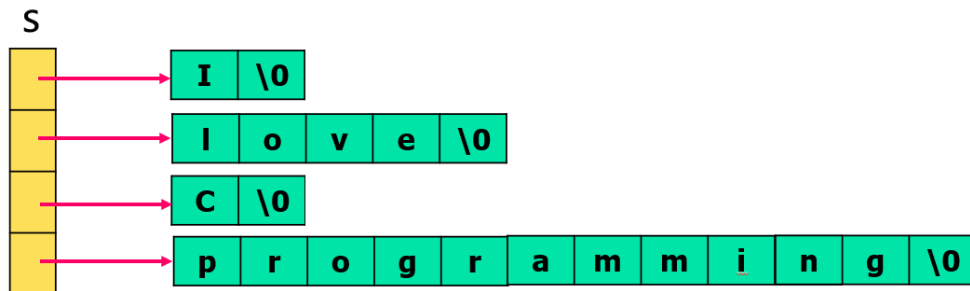
```
    }
```

```
    writeStrAry(s, STRNUM);    /*将上面输入的STRNUM个字符串反顺序输出 */
```

```
    for(i=0; i< STRNUM; i++) free(s[i]); /* 释放空间，分次申请的空间要逐个分别释放*/
```

```
    return 0;
```

```
}
```



```
void writeStrAry(const char *p[ ], int n)
```

```
    /*将 p 中的 n 个字符串从后往前输出*/
```

```
    {
```

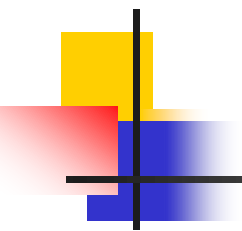
```
        while(--n>=0) printf("%s\n", p[n]);
```

```
    }
```



本章小结

- 指针的概念、声明、引用、间访。
- 指针的运算：算术运算、赋值运算、关系运算。
- 多重指针的概念和使用。
- 指针作为函数的参数：址传递。
- 指针数组：以指针为元素的数组。
- 指针函数和函数指针的概念和应用。
- 指向数组的指针
- 用指向数组基本元素的指针访问多维数组。
- 用“数组类型”和“超数组”观点理解多维数组。

- 
- 带参数的main函数：提供了一种操作系统向所调用函数传递参数的途径，掌握命令行、参数个数、由字符指针数组指向的各个参数等概念和编程方法。
 - 函数指针：是以函数的入口地址为其值的指针变量，可以通过函数指针来调用它所指的函数。
 - 指针函数：返回值是指针类型值的函数，通过指针函数可以间接解决函数返回多值的问题。

实战题

1. 以下声明语句中正确的是 【 】（单选）

- A、int x[10],p=x; B、int x[10][20],(*p)[10]=x;
C、int x[5][6],*p[6]=x; D、int x[1][2],(*p)[2]=x;

2. 对数组元素a[i][j][k]，下面正确的表示有 【 】（多选）

- A、(*(a+i))[j][k] B、*(*(a+i)+j)+k
C、*(*(a+i)[j]+k) D、*(a[i][j]+k)

3. 判断改错题：定义一个指向常量的常指针的声明如下：

```
const char *p="abcd" ;
```

4. 设p是函数指针，所指向的函数有两个整型参数且返回一个指向有5个元素的字符数组的指针，请写出相应的声明语句。

```
int (*)[5](*p)(int,int);
```

```
typedef char (*myptrtype)[5];
```

```
myptrtype(*p)(int,int);
```

请写出下面程序的运行结果。

```
#include<stdio.h>
```

```
#define M 5
```

```
#define N 2
```

```
void main()
```

```
{
```

```
    static a[M];
```

```
    int i,n,k=M-1,*p1,*p2;
```

```
    p1=p2=&a[k];
```

/*p1、 p2指向a的最后一个元素*/

```
    for(n=0;n<k;n++) {
```

```
        for(i=0;i<N;++i) {
```

```
            while(1) {
```

```
                if(++p1>p2) p1=a;
```

```
                if(!*p1) break;
```

```
            }
```

```
        }
```

```
        *p1=-1;
```

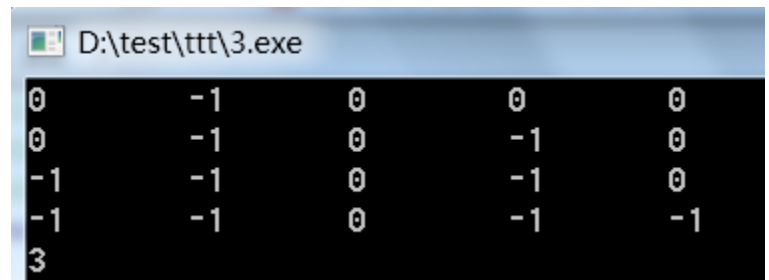
```
        for(i=0;i<M;++i) printf("%d\t",a[i]);
```

```
        printf("\n");
```

```
    }
```

```
    for(i=0;i<M;++i) if(!a[i]) printf("%d\n",i+1);
```

```
}
```



```
D:\test\ttt\3.exe
0      -1      0      0      0
0      -1      0      -1     0
-1     -1      0      -1     0
-1     -1      0      -1    -1
3
```